

碳化硅与硅外延层厚度的确定

摘要

碳化硅外延层厚度是决定功率器件耐压与可靠性的关键参数。本文针对利用红外干涉光谱反演碳化硅外延层厚度的问题，建立双光束干涉模型与双角度联合厚度反演算法，进一步推广为多光束干涉模型，实现对衬底吸收所引入系统偏差的评估与补偿，利用同一晶圆片在两个入射角下的实测光谱完成求解与可靠性检验。

针对问题一，将晶圆片抽象为空气、外延层与衬底构成的三层介质结构，由 Snell 定律与几何光程差推导两束反射光的相位关系，结合半波损失抵消分析及 s/p 偏振菲涅尔系数，借助 Stokes 倒逆关系建立反射率与厚度、折射率之间的显式函数关系；针对折射率随波长变化的特性，分别引入 Cauchy 色散公式与 Sellmeier 单振子公式，得到两种厚度确定模型，并将厚度反演归结为非线性最小二乘问题。

针对问题二，设计完整的光谱预处理与参数反演流程。采用滑动窗口 FFT 定量确定低波数异常区的裁剪分界点，经等间距重采样、Hampel 滤波、AsLS 基线校正与 Savitzky-Golay 平滑提取纯干涉振荡信号；构造两个入射角共享厚度与色散系数的联合非线性最小二乘目标，以 FFT 与差分进化提供参数初值，用 L-M 算法精调厚度。计算得到 Cauchy 模型厚度 $7.4755\ \mu\text{m}$ 、Sellmeier 单振子模型厚度 $7.4791\ \mu\text{m}$ ，两者相对偏差约 0.05%，95% 置信区间宽度小于 $0.012\ \mu\text{m}$ ；AIC 与 BIC 一致表明 Sellmeier 单振子模型更优。可靠性检验表明，加入 5% 高斯噪声后厚度漂移小于 0.1%，算法对随机噪声稳健。

针对问题三，将双光束模型推广为多光束干涉，通过对逐次反射复振幅等比级数求和导出 Airy 反射率公式，并从振幅收敛比、光源相干性、表面平行度、吸收损耗与光斑重叠等方面推导多光束干涉的必要条件，结合机制诊断仿真量化衬底吸收对厚度的系统偏差。对硅片，采用双光束与 Airy 双模型拟合、界面反射率估计与残差谐波检验，以 AIC/BIC 模型比选作为最终判据，建立 Airy 反射率、Sellmeier 外延层与 Drude 衬底色散相结合的双角度联合反演模型，以 FFT 与差分进化提供初值，用 L-M 算法精拟合。结果表明：硅片 $\Delta\text{AIC} = +134.85$ ，出现多光束干涉，修正后厚度 $3.4030\ \mu\text{m}$ ，95% 置信区间 $[3.4013, 3.4047]\ \mu\text{m}$ ；碳化硅片 $\Delta\text{AIC} = -15.24$ 、 $\rho = 0.0035$ ，未出现多光束干涉，无需修正。该方法可识别小界面反射率下由衬底吸收复相位引起的“隐式”多光束效应，将判定从定性观察升级为可检验的定量决策。

关键字： 碳化硅; 色散模型; 多光束干涉; L-M 算法; 非线性最小二乘; Airy 公式

一、问题重述

1.1 问题背景

碳化硅是第三代半导体材料的代表，其外延层厚度对器件击穿电压、导通电阻与长期可靠性有重要影响，是外延材料质量控制的关键参数。红外干涉法利用外延层与衬底因掺杂浓度不同而折射率不同的特点，使红外光在两界面反射的两束光形成干涉条纹；条纹形态由波长、外延层折射率与入射角共同决定，故可由实测干涉光谱反推外延层厚度。由于外延层折射率随波长与掺杂浓度变化，建模时必须计入折射率色散。红外光在外延结构中的双光束干涉过程如图 1 所示。

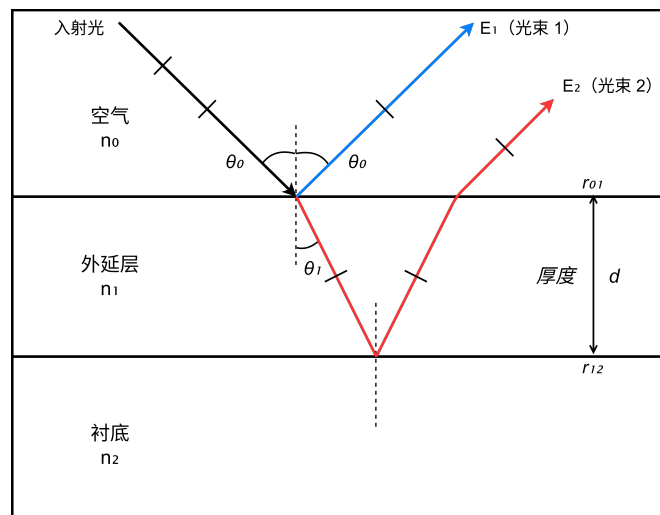


图 1 红外光在空气、外延层与衬底三层介质中的双光束干涉示意图

1.2 问题提出

本问题属于基于红外干涉光谱的外延层厚度反演问题，可归结为物理建模、参数反演与可靠性验证三类任务。

针对问题一，在只考虑外延层上表面一次反射、透射以及衬底表面一次反射、透射的近似条件下，建立反射率 $R(\nu)$ 与厚度 d 、外延层折射率 $n_1(\nu)$ 之间的显式数学模型，为厚度反演提供正模型。

针对问题二，基于问题一的数学模型，设计由实测反射率光谱反演厚度 d 的数值算法；利用附件两个不同角度的数据对同一碳化硅晶圆片的实测光谱给出计算结果，并检验结果的可靠性。

针对问题三，推导光波在外延层界面与衬底界面多次反射、透射产生多光束干涉的必要条件，分析多光束干涉对外延层厚度计算精度的影响；据此判定附件 3 与附件 4 中

硅晶圆片测试结果是否出现多光束干涉，建立硅外延层厚度计算模型与算法并给出结果；若多光束干涉也影响碳化硅晶圆片测试结果，需设法消除其影响并给出修正结果。

二、问题分析

2.1 问题一分析

问题一旨在建立三层介质结构下的双光束干涉模型，刻画红外反射光谱与外延层厚度、折射率之间的映射关系。仅考虑上下界面各一次反射透射的两束相干光，需要结合 **Snell 定律** 推导几何相位差，判断半波损失的抵消情况；通过 **菲涅尔公式** 分别计算 s、p 偏振反射系数并做平均；利用 **Stokes 倒逆关系** 化简透射振幅；由于折射率是变化的，因此引入 **色散模型** 以简化折射率随波数的变化；以上模型为后续问题的求解提供理论依据。

2.2 问题二分析

问题二基于问题一的双光束模型，利用两组不同入射角实测光谱完成外延层厚度反演，并开展多维度可靠性评估。两套数据来自同一块样品，厚度、色散参数保持不变，可构建双角度联合拟合约束提升辨识能力。原始光谱存在低波数畸变、基线漂移、非等间隔采样等问题，需要针对以上问题的进行数据预处理。反演存在两处数值难点：非线性最小二乘对初值敏感，对此采用 **FFT**、**差分进化** 获取初值；厚度维度目标函数存在多周期分支，对此采用剖面扫描结合 **L-M** 算法规避解跳变。求解完成后，从拟合优度、参数置信区间、抗噪声仿真、双角度一致性、残差时域与频域结构开展检验；若残差出现周期性振荡，可为后续多光束效应分析提供依据。

2.3 问题三分析

问题三在前两问的基础上，将干涉模型由双光束推广到多光束，要求分析多光束干涉的产生条件及其对厚度计算精度的影响，并结合两类晶圆片的实测光谱选择合适的模型，得到更精确的厚度结果。具体需要解决三个问题：其一，推导多光束干涉的必要条件，并分析其对厚度计算精度的影响；其二，判定附件 3、4 硅片是否出现多光束干涉，建立硅外延层厚度的数学模型与算法并给出计算结果；其三，回检附件 1、2 碳化硅数据是否需要修正，若需要则消除影响并重新给出厚度。

针对上述问题，首先对问题一的双光束模型进行修正，将逐次反射的复振幅按等比级数累加，推广为 **Airy** 多光束模型，并据此推导必要条件与精度影响；随后按问题二的流程对附件 3、4 的光谱数据进行预处理，采用频域分析获取厚度初值，用非线性优化对双光束与多光束模型分别拟合，通过模型比选定量判定多光束干涉是否出现，并完成

可靠性评估。最后将同一套判据应用于附件 1、2，检验问题二给出的碳化硅厚度结论是否需要修正，从而保证两类材料的判定口径一致、结论相互印证。

下图为全文技术路线图：

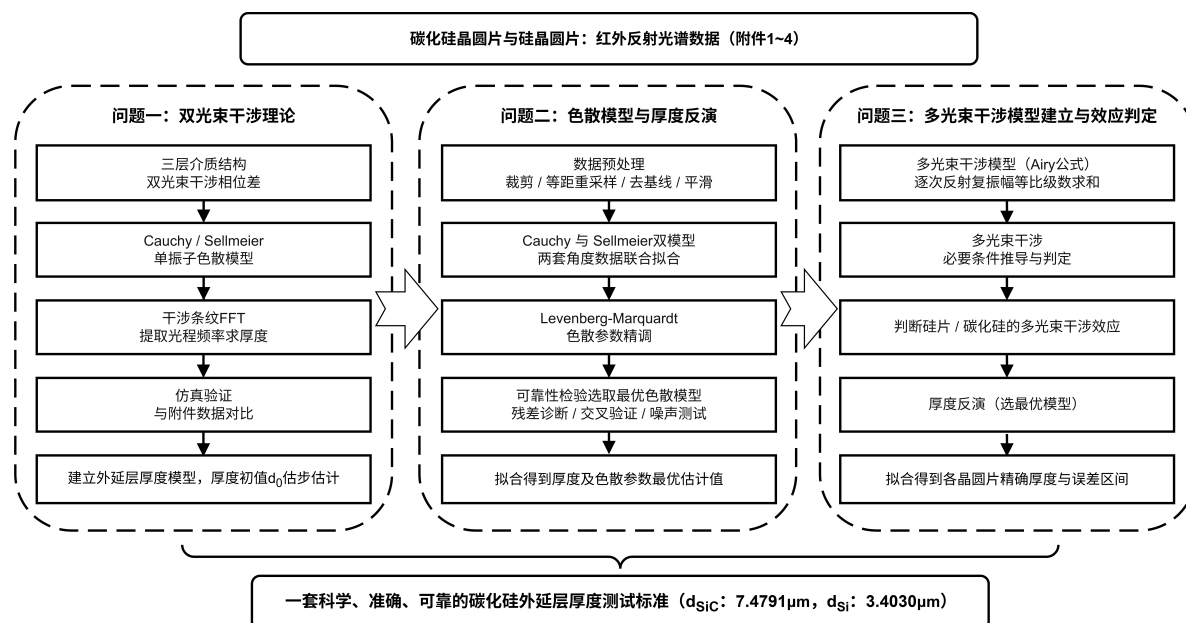


图 2 整体技术路线图

三、模型假设

(1) 介质与界面假设：空气、外延层、衬底为均匀各向同性透明介质；上下界面近似平行，忽略表面粗糙散射。

(2) 平面波假设：入射光为单色平面波，入射角为 θ_0 ，忽略光源有限孔径效应。

(3) 折射率大小假设：折射率满足 $n_0 < n_1 < n_2$ ，用于判定半波损失条件。

(4) 折射率色散假设：外延层折射率 n_1 随波长变化，采用色散模型描述；衬底折射率在测量波段取恒定值（文献）。

(5) 数据一致性假设：两组入射角光谱来自同一样品，厚度与色散参数不变；测量噪声为零均值高斯白噪声。

(6) 多光束叠加假设：问题三中光在外延层内多次往返反射，逐次反射复振幅构成等比级数；界面反射率较低时级数收敛，可用 Airy 公式描述总反射率。

(7) 理想平行平面假设：多光束干涉成立以光斑内厚度高度均匀为前提，上下表面近似平行、粗糙度远小于波长。

(8) 硅衬底色散假设：硅片重掺杂衬底的自由载流子响应用 Drude 色散模型描述；外延层为轻掺杂硅，在中红外波段近似无色散。

四、符号说明

本文涉及的主要符号见表 1。

表 1 本文主要符号说明

符号	说明	单位
n_i	第 i 层介质折射率	无量纲
d	外延层厚度	μm
θ_i	第 i 层传播角	$^\circ$
ν	波数, $\nu = 1/\lambda$	cm^{-1}
λ	真空波长	μm
δ	反射光相位差	rad
r_{ij}	第 i 到 j 层界面反射系数	无量纲
t_{ij}	第 i 到 j 层界面透射系数	无量纲
E_m	第 m 束反射光复振幅	相对值
R	反射率	% 或无量纲
f_0	干涉条纹光程频率	cm
β	色散系数向量	依模型而定
a_i, b_i	第 i 角度校准增益与偏移	无量纲
ρ	相邻反射光振幅收敛比	无量纲
n'_i, κ_i	复折射率实部与虚部	无量纲
$n_\infty, \omega_p, \gamma$	Drude 模型高频极限折射率、等离子波数与碰撞波数	无量纲、 cm^{-1}
H_m	第 m 次谐波信噪比	无量纲

五、问题一的模型的建立和求解

5.1 模型光学原理

5.1.1 三层介质结构与物理模型

将晶圆片抽象为空气层 (n_0)、外延层 (n_1 , 厚度 d)、衬底 (n_2) 构成的三层结构, 折射率满足 $n_0 < n_1 < n_2$ 。波数为 ν 的红外光以入射角 θ_0 入射上表面: 一部分在上表面反射形成光束 1, 另一部分以折射角 θ_1 进入外延层、经下表面反射后透射回空气形成光束 2, 两束光在空气侧叠加产生干涉条纹。建模采用模型假设第 (1) 至 (5) 条。

5.1.2 光程差与相位差

由 Snell 定律，折射角 θ_1 满足

$$n_0 \sin \theta_0 = n_1 \sin \theta_1, \quad (1)$$

式中 n_0 、 n_1 为空气与外延层折射率， θ_0 为入射角； θ_1 非独立变量。光束 2 往返光程为 $2n_1d$ ，投影到反射波法线方向（乘 $\cos \theta_1$ ）得几何光程差

$$\Delta = 2n_1d \cos \theta_1, \quad (2)$$

其中 d 为外延层厚度。相位差为

$$\delta = \frac{2\pi}{\lambda} \Delta = \frac{4\pi n_1 d \cos \theta_1}{\lambda} = 4\pi n_1 d \cos \theta_1 \nu, \quad (3)$$

其中 λ 为真空波长， $\nu = 1/\lambda$ 为波数。 δ 随 ν 线性增长，故反射率呈周期变化，即干涉条纹的来源。数值计算中 d 以 μm 、 ν 以 cm^{-1} 计时，相位差取 $\delta = 4\pi n_1 d \cos \theta_1 \nu / 10^4$ 。

5.1.3 半波损失分析

光从光疏介质射向光密介质时，反射光产生 π 附加相位。两束反射光的半波损失情况见表 2。

表 2 两束反射光的半波损失分析

反射事件	发生界面	折射率变化方向	是否发生半波损失
光束 1（上表面反射）	空气-外延层	$n_0 \rightarrow n_1$ （光疏 $\rightarrow$ 光密）	是
光束 2（下表面反射）	外延层-衬底	$n_1 \rightarrow n_2$ （光疏 $\rightarrow$ 光密）	是

由表 2，两束反射光各经历一次半波损失，**两次附加相位 π 相互抵消**，实际相位差仍为式 (3) 的 δ 。若 $n_2 < n_1$ ，干涉项变为 $-\cos \delta$ ，仅平移条纹、不改变周期，不影响厚度反演；碳化硅衬底掺杂更高（ $n_2 > n_1$ ），本文取抵消情形。

5.1.4 菲涅尔公式与界面振幅系数

斜入射时反射系数与偏振有关。记 n_a 、 n_b 为入射侧、折射侧折射率， θ_a 、 θ_b 为入射角、折射角（ θ_b 由 Snell 定律确定），则 s 偏振（电场垂直入射面）与 p 偏振（电场平行入射面）的振幅反射系数分别为

$$\begin{cases} r_{ab}^{(s)} = \frac{n_a \cos \theta_a - n_b \cos \theta_b}{n_a \cos \theta_a + n_b \cos \theta_b}, \\ r_{ab}^{(p)} = \frac{n_b \cos \theta_a - n_a \cos \theta_b}{n_b \cos \theta_a + n_a \cos \theta_b}, \end{cases} \quad (4)$$

两偏振表达式不同，斜入射必须区分。本文涉及空气-外延层界面 (r_{01} , 透射系数 t_{01} 、 t_{10}) 与外延层-衬底界面 (r_{12})。非偏振入射光取两偏振平均：

$$R = \frac{R^{(s)} + R^{(p)}}{2}, \quad (5)$$

其中 $R^{(s)} = |r^{(s)}|^2$ 、 $R^{(p)} = |r^{(p)}|^2$ 为两偏振强度反射率。

5.1.5 双光束复振幅合成与反射率

设入射光复振幅为 E_0 ，两束反射光复振幅分别为

$$\begin{cases} E_1 = r_{01} E_0, \\ E_2 = \tau r_{12} e^{i\delta} E_0, \end{cases} \quad (6)$$

其中 r_{01} 、 r_{12} 为上、下界面振幅反射系数， $\tau = t_{01}t_{10}$ 为上界面往返透射系数乘积， δ 为式 (3) 的相位差。光束 2 振幅因子为 τr_{12} 而非单纯的 r_{12} 。无吸收界面满足 Stokes 倒逆关系

$$\tau = 1 - r_{01}^2, \quad (7)$$

即透射往返的强度损失等于单次反射的强度贡献。代入式 (6)，合成振幅反射系数为

$$r = \frac{E_1 + E_2}{E_0} = r_{01} + (1 - r_{01}^2) r_{12} e^{i\delta}, \quad (8)$$

取模平方得反射率

$$\begin{aligned} R(\nu) &= |r|^2 \\ &= r_{01}^2 + (1 - r_{01}^2)^2 r_{12}^2 + 2 r_{01} r_{12} (1 - r_{01}^2) \cos \delta, \end{aligned} \quad (9)$$

其中前两项为两束光各自的强度贡献（直流项），第三项为干涉项。 r_{01} 、 r_{12} 按式 (4) 计算并对两偏振取平均； $n_0 < n_1 < n_2$ 使半波损失抵消，干涉项取 $+\cos \delta$ 。条纹周期与极值位置由 δ 决定，厚度 d 的信息即包含其中。

5.1.6 折射率色散模型

若 n_1 视为常数，条纹周期只能确定乘积 $n_1 d$ ，厚度与折射率无法分离，须引入色散模型 $n_1(\lambda)$ 。模型 A 采用 Cauchy 公式：

$$n_1(\lambda) = A + \frac{B}{\lambda^2} + \frac{C}{\lambda^4}, \quad (10)$$

其中 λ 以 μm 计, A 为常数项, B 、 C 为高阶色散系数; 数值实现按 $\lambda = 10^4/\nu$ 换算。模型 B 采用 Sellmeier 单振子公式:

$$n_1^2(\lambda) = 1 + \frac{B\lambda^2}{\lambda^2 - C}, \quad (11)$$

其中 B 为振子强度, C 为谐振波长的平方 (小于测量波段波长平方, 保证 n_1 为实数)。2.5 ~ 6.4 μm 窄波段内仅一个振子可辨识, 故取单振子形式。

5.1.7 两种厚度确定模型

将式 (10) 或式 (11) 的折射率代入式 (9), 两模型反射率统一写为

$$R^{(D)}(\nu) = r_{01}^2 + (1 - r_{01}^2)^2 r_{12}^2 + 2 r_{01} r_{12} (1 - r_{01}^2) \cos \left(\frac{4\pi n_1^{(D)}(\nu) d \cos \theta_1 \nu}{10^4} \right), \quad (12)$$

其中上标 $D \in \{C, S\}$ 区分 Cauchy 与 Sellmeier 模型, $n_1^{(D)}(\nu)$ 由式 (10) 或式 (11) 给出; r_{01} 、 r_{12} 按两偏振计算后取平均, $\cos \theta_1$ 由式 (1) 确定。待定参数见表 3。

表 3 两种模型的待定参数

模型	待定参数	说明
Cauchy 模型	d, A, B, C	厚度 d 为核心量, 其余为色散系数
Sellmeier 模型	d, B, C	厚度 d 为核心量, 其余为色散系数

5.2 模型求解思路

由式 (9) 的干涉项 $\cos \delta$ 可知, 反射率随波数 ν 以光程频率

$$f_0 = \frac{2n_1 d \cos \theta_1}{10^4}, \quad (13)$$

周期振荡, 故当折射率 n_1 已知时, 由条纹频率反解式 (13) 即可确定厚度。当外延层存在色散时, n_1 随波长变化, 厚度与色散系数须联合确定。给定实测反射谱 $R^{\text{obs}}(\nu)$, 厚度与色散系数由最小化理论反射率与实测值的残差平方和确定:

$$\min_{d, \beta} S = \sum_{k=1}^N \left[R^{(D)}(\nu_k; d, \beta) - R^{\text{obs}}(\nu_k) \right]^2, \quad (14)$$

其中 β 为色散系数向量, N 为光谱点数。式 (14) 是非线性最小二乘问题; 数值计算取 $n_0 = 1.0$ 、 $n_2 = 2.65$ 作为已知量。

六、问题二的模型的建立和求解

6.1 数据预处理

6.1.1 数据概况与原始光谱观察

附件 1 与附件 2 是同一碳化硅晶圆片在入射角 10° 与 15° 下的红外反射率光谱，第 1 列为波数 ν (cm^{-1})、第 2 列为反射率 R (%), 波数范围约 $399.67 \sim 4000.12 \text{ cm}^{-1}$ 、步长约 0.482 cm^{-1} 。图 3 给出原始红外反射光谱。

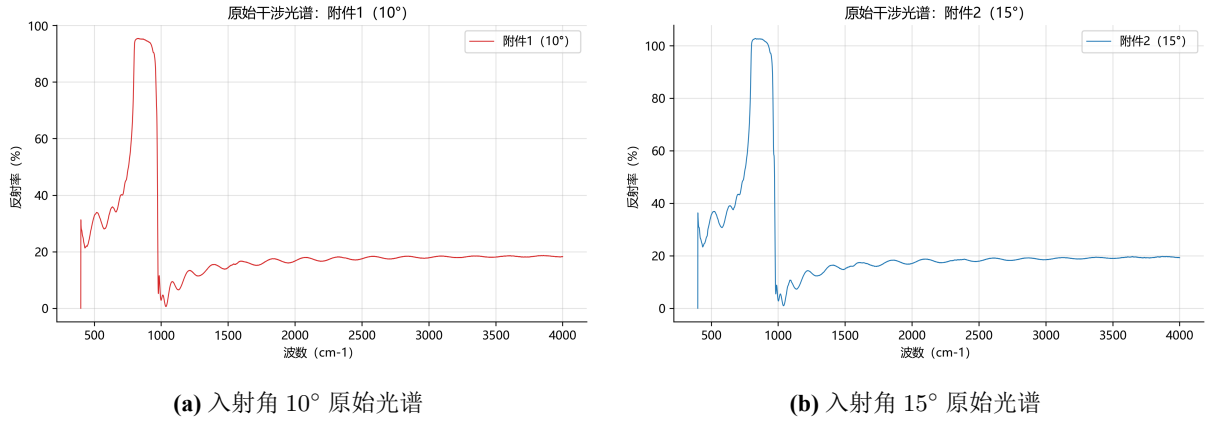


图 3 附件 1 与附件 2 的原始红外反射光谱

两套数据形态一致：低波数区（约 $< 1550 \text{ cm}^{-1}$ ）存在强背景（Reststrahlen 带）与基线漂移，高波数区呈现清晰干涉条纹；两角度条纹周期因 $\cos \theta_1$ 不同而略有差异，可为联合拟合提供独立信息。低波数异常区会污染 FFT 快速傅里叶变换，须先进行裁剪。

6.1.2 低波数异常区间的定量裁剪

忽略色散缓变影响时，式 (9) 近似为

$$R(\nu) \approx \text{DC} + A_c \cos(2\pi f_0 \nu), \quad (15)$$

其中 DC 为直流项， A_c 为振荡幅度， f_0 为式 (13) 定义的光程频率（此处以平均折射率 $\bar{n}$ 近似 n_1 ）， d 以 μm 计。有效信号能量集中于 f_0 附近，窗口落入异常区时主峰消失或淹没于噪声。据此用滑动窗口 FFT（短时傅里叶变换）定量确定裁剪分界点：对窗口中心 ν_c ，先去趋势（1000 点移动平均）、加汉宁窗做 FFT，计算主频带与低频残留能量比

$$\rho(\nu_c) = \frac{E_{\text{band}}(f_0; \nu_c)}{E_{\text{low}}(f_0; \nu_c)}, \quad (16)$$

其中 E_{band} 为主频带能量 ($f_0 \pm 4$ 个频率 bin), E_{low} 为低频残留能量 ($f < f_0/2$), 窗口参数见表 4。裁剪分界点定义为 $\rho(\nu_c)$ 首次达到峰值一定比例处:

$$\nu_{\text{cut}} = \min \{ \nu_c : \rho(\nu_c) \geq \eta \rho_{\text{max}} \}, \quad (17)$$

其中 ρ_{max} 为比值峰值, $\eta = 0.3$ 为阈值。得 $\nu_{\text{cut}} = 1556.755 \text{ cm}^{-1}$, 两附件共用, 裁剪后保留 $\nu \in [\nu_{\text{cut}}, \nu_{\text{max}}]$ (见图 4)。

表 4 滑动窗口 FFT 裁剪参数

参数	取值	含义
去趋势窗口	1000 点	移动平均
主频估计区间	$[1500, 4000] \text{ cm}^{-1}$	估干涉主频 f_0
窗口长度	1200 点	覆盖不少于 2 个条纹周期
滑动步长	200 点	相邻窗口重叠
主频带	$f_0 \pm 4$ 个频率 bin	主频带能量 E_{band}
低频带	$f < f_0/2$	低频残留能量 E_{low}
阈值 η	0.3	比值达到峰值的比例

6.1.3 等间距重采样

FFT 要求严格等距采样, 故在裁剪区间内按固定步长 $\Delta\nu$ 线性插值重采样:

$$R^{\text{res}}(m\Delta\nu) = R(\nu_k) + \frac{R(\nu_{k+1}) - R(\nu_k)}{\nu_{k+1} - \nu_k} (m\Delta\nu - \nu_k), \quad (18)$$

其中 m 为采样序号, $R(\nu_k)$ 、 $R(\nu_{k+1})$ 为相邻原始点, $\Delta\nu = 0.482 \text{ cm}^{-1}$ 为原始中位步长。点数 $N = \lfloor (\nu_{\text{max}} - \nu_{\text{cut}}) / \Delta\nu \rfloor + 1$, 得每角度 **5071** 点、合计 **10142** 点。

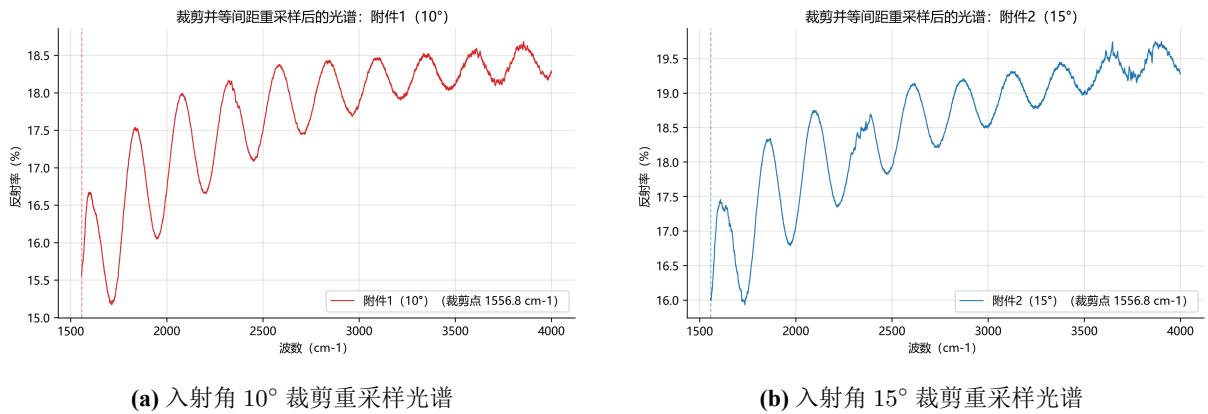


图 4 裁剪并等间距重采样后的反射率光谱

6.1.4 去异常、去基线与平滑

预处理顺序为 Hampel 滤波、AsLS 基线校正、Savitzky–Golay 平滑、去直流，顺序不可颠倒。

Step 1: Hampel 滤波剔除孤立异常值。对窗口内每点 x_i ，当 $|x_i - \text{med}| > t\sigma_{\text{MAD}}$ 时以中位数替换，其中 med 、 σ_{MAD} 分别为窗口中位数与绝对中位差；实现取窗口大小为 11，阈值 $t = 3.0$ 。

Step 2: AsLS 基线校正。对光谱 R_i 拟合基线 z_i ，最小化

$$\min_{\mathbf{z}} \sum_i w_i (R_i - z_i)^2 + \lambda \sum_i (\Delta^2 z)_i^2, \quad (19)$$

其中权重 $w_i = p(R_i > z_i)$ 或 $w_i = 1 - p(R_i \leq z_i)$ ，取 $p = 0.01$ 使基线贴附光谱下包络；第二项对二阶差分 $\Delta^2 z$ 施加平滑惩罚， $\lambda = 10^5$ ，迭代 10 次，校正后光谱记为 $R^{\text{res}} = R_i - z_i$ 。

Step 3: Savitzky–Golay 平滑。采用窗口内多项式最小二乘拟合，窗口长度取 15，多项式阶数取 3，在降噪的同时保留干涉振荡细节。

Step 4: 去直流处理。减去信号均值，使信号围绕 0 振荡，与理论干涉项形式保持一致。经过上述流程处理后的信号为幅度约 $\pm 0.36\%$ 的纯干涉振荡（图 5）。

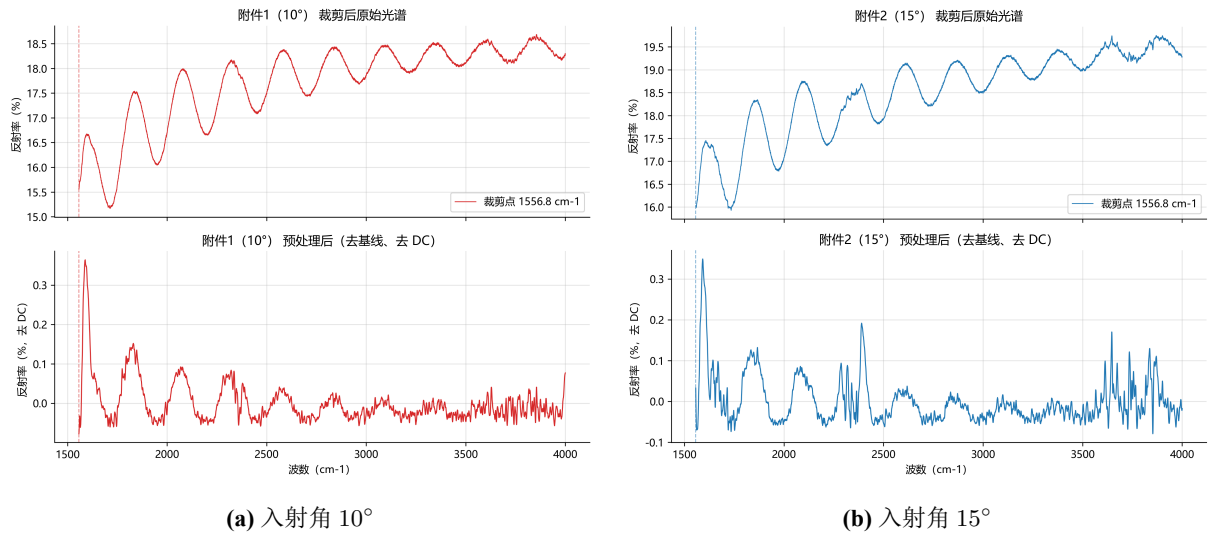


图 5 预处理前后光谱对比

6.2 厚度反演算法

6.2.1 反演问题的形式化

预处理后的实测信号为去直流振荡项，理论模型也取式 (9) 的振荡部分。对第 i 个入射角，理论振荡项为

$$R^{\text{osc}}(\nu; \theta_0^{(i)}, d, \beta) = 2 r_{01} (1 - r_{01}^2) r_{12} \cos\left(\frac{4\pi n_1(\nu; \beta) d \cos \theta_1 \nu}{10^4}\right), \quad (20)$$

其中 $\theta_0^{(i)}$ 为第 i 入射角, r_{01} 、 r_{12} 按两偏振计算取平均, $n_1(\nu; \beta)$ 为式 (10) 或式 (11) 的色散折射率; 式 (9) 的直流项已被预处理去除。理论振荡与实测间允许仿射校准, 以反映仪器增益与残余直流:

$$R_{i,k}^{\text{obs}} \approx a_i R^{\text{osc}}(\nu_{i,k}; \theta_0^{(i)}, d, \beta) + b_i, \quad (21)$$

其中 $R_{i,k}^{\text{obs}}$ 为第 i 角度第 k 点实测值, (a_i, b_i) 为该角度增益与偏移, 对固定 d, β 闭式求解、不进入非线性优化器。

同一晶圆片在两入射角下测量, 厚度 d 与色散系数 β 跨角度共享。构造双角度联合残差平方和:

$$\min_{\theta} S(\theta) = \sum_{i=1}^2 \sum_{k=1}^{N_i} \left[a_i R^{\text{osc}}(\nu_{i,k}; \theta_0^{(i)}, d, \beta) + b_i - R_{i,k}^{\text{obs}} \right]^2, \quad (22)$$

其中 N_i 为第 i 角度数据点数, 优化变量 θ 对 Cauchy 模型为 (d, A, B, C) 、对 Sellmeier 单振子模型为 (d, B, C) 。双角度联合拟合参数自由度不增而数据量翻倍, 可显著提高参数可辨识度。

6.2.2 参数初值估计

非线性最小二乘对初值敏感。本文先对预处理信号做 FFT (Hann 窗、跳过直流、抛物线插值细化峰值) 得光程频率 $\hat{f}_0$, 再由式 (15) 反推厚度初值:

$$d_0 = \frac{\hat{f}_0 \times 10^4}{2 \bar{n} \cos \bar{\theta}_1}, \quad (23)$$

其中 $\bar{n} = 2.6$ 为平均折射率, $\bar{\theta}_1$ 按式 (1) 由 $\bar{n}$ 计算, $\hat{f}_0$ 以 cm 计。 d_0 受频率分辨率限制, 仅作优化初值; 本数据得初始厚度 $d_0 = 7.6346 \mu\text{m}$ 。

固定 $d = d_0$, 用差分进化在约束范围内搜索色散系数初值 β_0 (目标为式 (22)); 实现取种群 20、最大迭代 200、容差 10^{-8} 。参数约束见表 5。

表 5 参数约束范围

参数	约束范围	依据
Cauchy A	[1.5, 3.5]	碳化硅红外折射率量级
Cauchy B, C	[-1.0, 1.0]	高阶色散项, 可正可负
Sellmeier B	[0, 8]	振子强度为正
Sellmeier C	[0, 1]	谐振点不落入测试波段

得到 Cauchy 初值 (2.6792, -0.3404, 1.0)、Sellmeier 单振子初值 (5.0689, 1.0)。

6.2.3 剖面扫描与 Levenberg–Marquardt 精调

目标函数在厚度方向存在多周期分支，直接将 d 与色散系数一起交给梯度优化易跳入错误分支，故采用剖面扫描 + Levenberg–Marquardt (L-M) 两段式策略：

第一段在 $[d_0 - 0.5, d_0 + 0.5] \mu\text{m}$ 内以步长 $0.01 \mu\text{m}$ 扫描，对每个 d 固定厚度、仅优化 β （线性参数闭式消元），记录对应的残差平方和 $S(d)$ ；第二段在 $S(d)$ 最小处做抛物线插值，得亚网格精调厚度：

$$\hat{d} = \arg \min_d S(d). \quad (24)$$

物理约束： $n_0 < n_1 < n_2$ 时理论振荡系数为正，故线性增益须 $a > 0$ ， $a \leq 0$ 对应反相非物理解直接排除。L-M 为高斯–牛顿法与梯度下降法的自适应结合，迭代格式为

$$\begin{cases} (\mathbf{J}^\top \mathbf{J} + \lambda \text{diag}(\mathbf{J}^\top \mathbf{J})) \Delta \boldsymbol{\theta} = -\mathbf{J}^\top \mathbf{r}, \\ \boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + \Delta \boldsymbol{\theta}, \end{cases} \quad (25)$$

其中 $\mathbf{r}$ 为双角度残差向量， $\mathbf{J} = \partial \mathbf{r} / \partial \boldsymbol{\theta}$ 为雅可比矩阵， λ 为自适应阻尼（残差下降时减小、上升时增大）， $\text{diag}(\mathbf{J}^\top \mathbf{J})$ 改善病态。实现采用带边界的信任域反射算法，精度 10^{-12} 、最大求值 1000 次。

6.3 模型求解结果与分析

6.3.1 双色散模型拟合结果

按式 (22) 对两模型分别执行完整反演流程，结果见表 6，线性校准参数见表 7。

表 6 两模型拟合结果对比

指标	Cauchy 模型	Sellmeier 单振子模型
厚度 d (μm)	7.4755	7.4791
厚度标准差 (μm)	0.00255	0.00307
d 的 95% 置信区间 (μm)	[7.4705, 7.4805]	[7.4731, 7.4851]
色散系数	$A = 2.5177, B = 1.0,$ $C = 1.0$	$B = 5.3193, C = 1.0$
色散系数 95% 置信区间	$A \in [2.5133, 2.5221]$	$B \in [5.3096, 5.3291]$
RMSE (%)	0.04289	0.04276
R^2	0.4025	0.4059
AIC	-63863.7	-63924.1
BIC	-63813.2	-63880.7

表 7 两角度的线性校准参数

入射角	Cauchy (a, b)	Sellmeier (a, b)
10°	$(0.05803, -3.67 \times 10^{-4})$	$(0.05703, -2.98 \times 10^{-4})$
15°	$(0.05758, -5.88 \times 10^{-4})$	$(0.05671, -4.97 \times 10^{-4})$

主要结论如下。

第一，两模型厚度高度一致：Cauchy 模型 $d = 7.4755 \mu\text{m}$ 、Sellmeier 单振子模型 $d = 7.4791 \mu\text{m}$ ，相对偏差 0.05%，说明厚度对色散模型形式不敏感。拟合对比见图 6 与图 7。

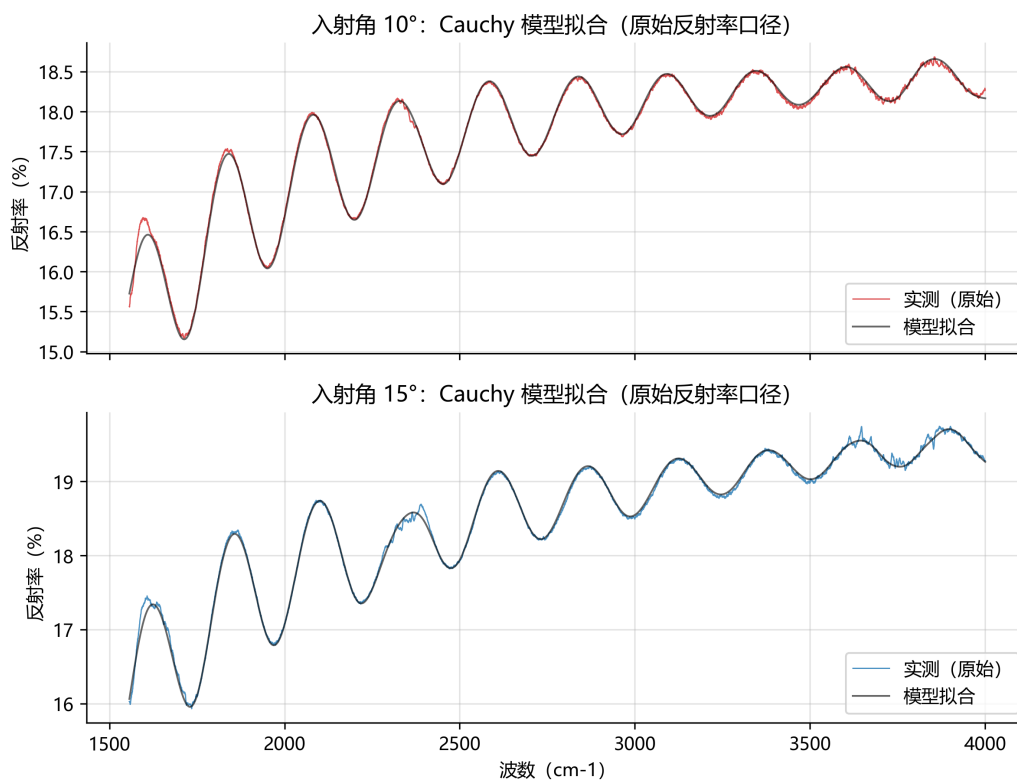


图 6 Cauchy 模型拟合对比

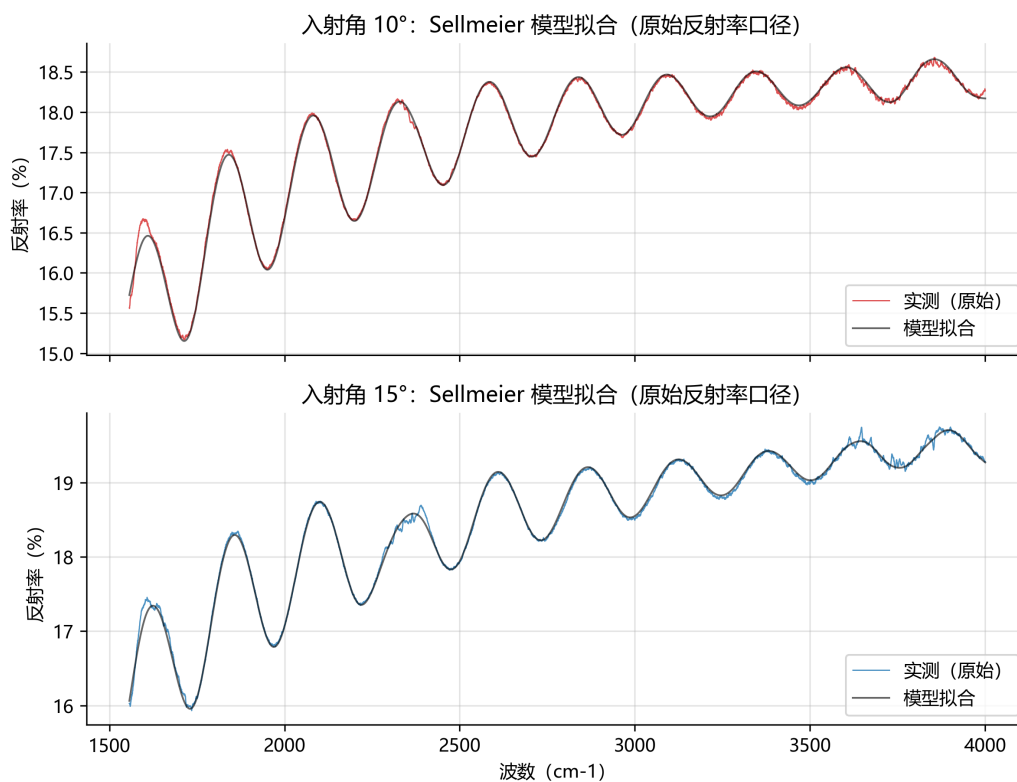


图 7 Sellmeier 单振子模型拟合对比

第二，折射率色散方面，由拟合系数得测量波段内 n_1 约为 2.54 ($\lambda = 6.4 \mu\text{m}$) 至 2.71 ($\lambda = 2.5 \mu\text{m}$)，呈正常色散（折射率随波长增大而减小），如图 8 所示，支撑折射率随波长变化的假设。

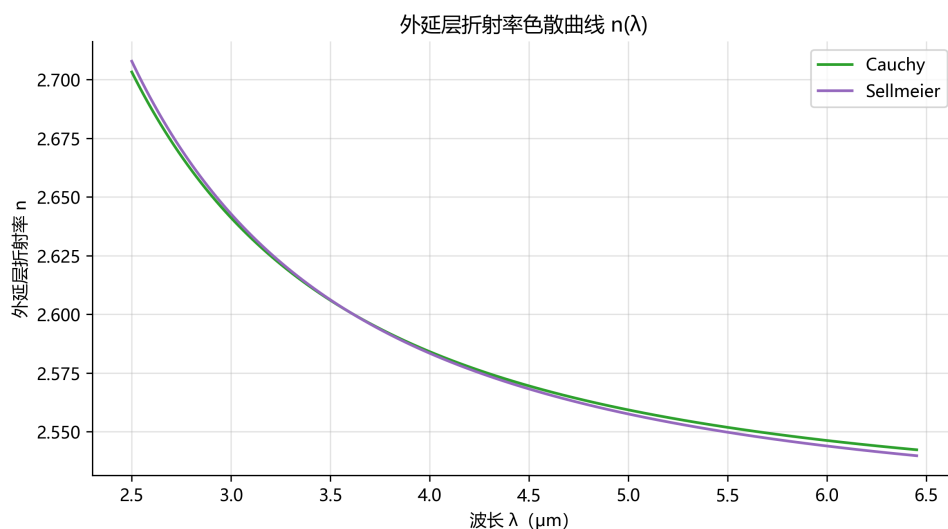


图 8 两种色散模型给出的外延层折射率随波长的变化曲线

第三，拟合优度方面，RMSE 均约 0.043%、很小，但 R^2 仅约 0.40 ~ 0.41 且残差呈周期结构（图 9、图 10），说明双光束模型抓住了干涉主振荡，但遗漏了与条纹同周期的系统成分，偏差主要来自模型结构误差而非随机噪声。

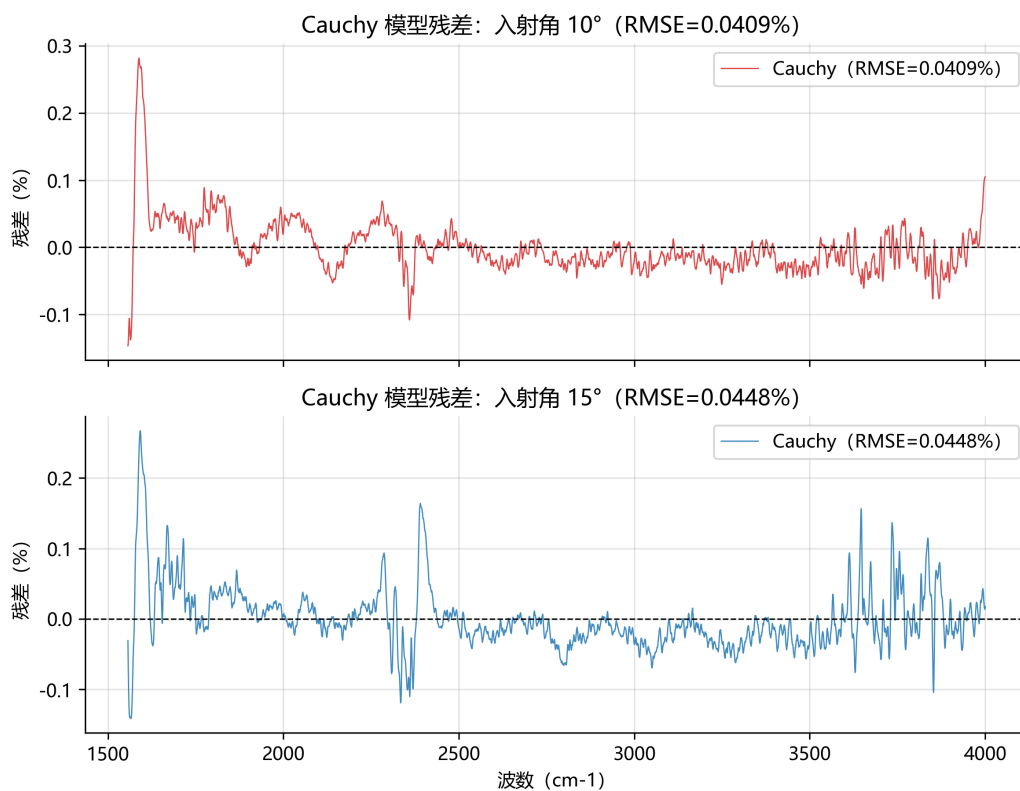


图 9 Cauchy 模型残差随波数的分布

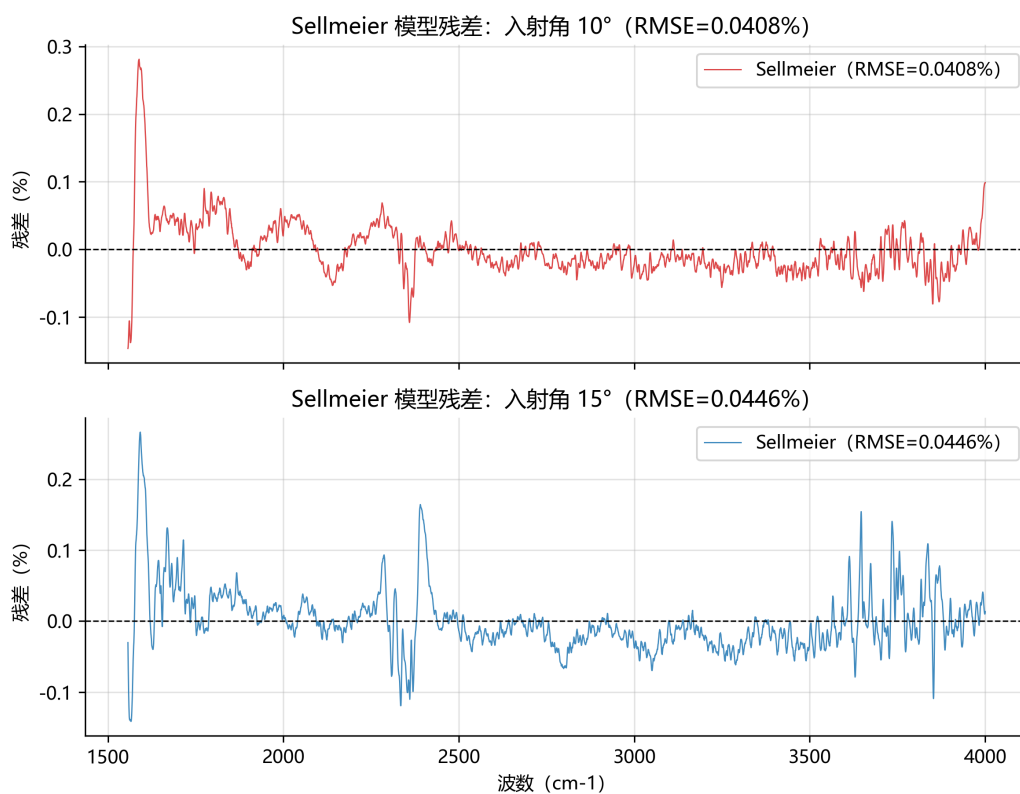


图 10 Sellmeier 单振子模型残差随波数的分布

6.4 可靠性检验与分析

6.4.1 拟合优度与残差统计

定义残差向量 $\mathbf{r} = (r_1, \dots, r_N)^\top$ ，均方根误差与决定系数为

$$\text{RMSE} = \sqrt{\frac{S(\hat{\boldsymbol{\theta}})}{N}}, \quad R^2 = 1 - \frac{S(\hat{\boldsymbol{\theta}})}{\sum_{k=1}^N (R_k^{\text{obs}} - \overline{R^{\text{obs}}})^2}, \quad (26)$$

其中 $N = 10142$, $\overline{R^{\text{obs}}}$ 为实测信号均值。两模型 RMSE 分别为 0.04289% 与 0.04276%、残差均值接近零，说明模型对干涉主成分的拟合在反射率绝对量级上相当精确； R^2 偏低来自残差中的周期结构，属系统性模型误差。

6.4.2 参数不确定性与置信区间

最优解处残差方差估计为

$$\hat{\sigma}^2 = \frac{S(\hat{\boldsymbol{\theta}})}{N - p}, \quad (27)$$

其中 $p = n_\beta + n_{\text{lin}}$ 为估计参数总数： n_β 为色散系数个数（Cauchy 为 3、Sellmeier 单振子为 2）， $n_{\text{lin}} = 4$ 为两角度共 4 个线性校准参数；厚度 d 由剖面扫描确定，作为公共参数不参与模型间比较。色散系数协方差矩阵为

$$\Sigma_\beta = \hat{\sigma}^2 (\mathbf{J}^\top \mathbf{J})^{-1}, \quad (28)$$

95% 置信区间由 $\hat{\beta}_j \pm t_{0.975, N-p} \hat{\sigma}_{\beta_j}$ 给出。结果见表 6：Cauchy 的 A 区间窄且远离边界， B 、 C 位于边界附近；Sellmeier 的 B 区间为 $[5.3096, 5.3291]$ ，辨识良好。

厚度 d 的不确定性由剖面目标函数曲率给出。将 $S(d)$ 在最小值附近抛物线拟合

$$S(d) \approx a_2 (d - \hat{d})^2 + S(\hat{d}), \quad (29)$$

其中 a_2 为曲率系数，则 $\sigma_{\hat{d}} = \sqrt{\hat{\sigma}^2 / a_2}$ 。两模型 $\sigma_{\hat{d}}$ 分别为 0.00255、0.00307 μm ，置信区间宽度约 $\pm 0.005 \mu\text{m}$ ，厚度估计精度高且相互一致。

6.4.3 抗噪声能力测试

对预处理光谱加入高斯噪声

$$R^{\text{noisy}} = R^{\text{obs}} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma_n^2), \quad \sigma_n = \eta \text{std}(R^{\text{obs}}), \quad (30)$$

其中 η 取 1% ~ 5%， $\text{std}(R^{\text{obs}})$ 为信号样本标准差。每等级重复 5 次、每次重走完整反演流程（抗噪试验中剖面扫描范围半宽取 $2.0\ \mu\text{m}$ 、步长取 $0.05\ \mu\text{m}$ 以控制计算量），结果见图 11。

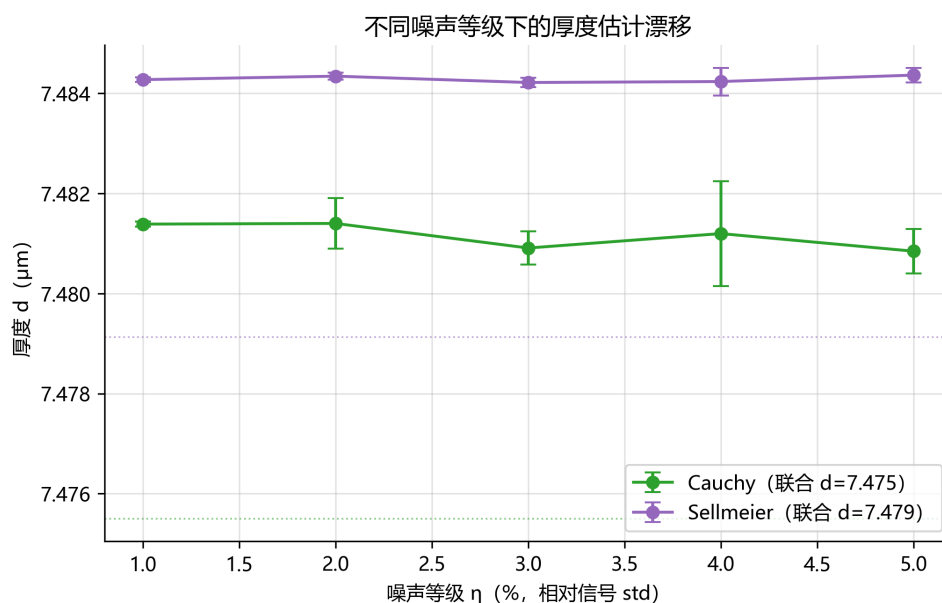


图 11 厚度随加入噪声等级的变化

以抗噪试验自身配置下的无噪声解（约 7.481 与 $7.484\ \mu\text{m}$ ）为基准，5% 噪声下厚度漂移小于 $0.005\ \mu\text{m}$ 、相对偏差小于 **0.1%**、标准差小于 $0.001\ \mu\text{m}$ 。抗噪试验采用较粗扫描步长，其无噪声解与表 6 精化解相差约 $0.006\ \mu\text{m}$ ，远小于测量不确定度。随机噪声经预处理链路抑制后几乎不影响厚度估计。

6.4.4 双角度一致性交叉验证

将附件 1 与附件 2 分别独立拟合，与联合拟合对比，结果见表 8 与图 12。

表 8 双角度一致性交叉验证结果

模型	联合拟合 (μm)	仅 10° (μm)	仅 15° (μm)
Cauchy	7.4755	7.7294	7.6231
Sellmeier 单振子	7.4791	7.5226	7.4197

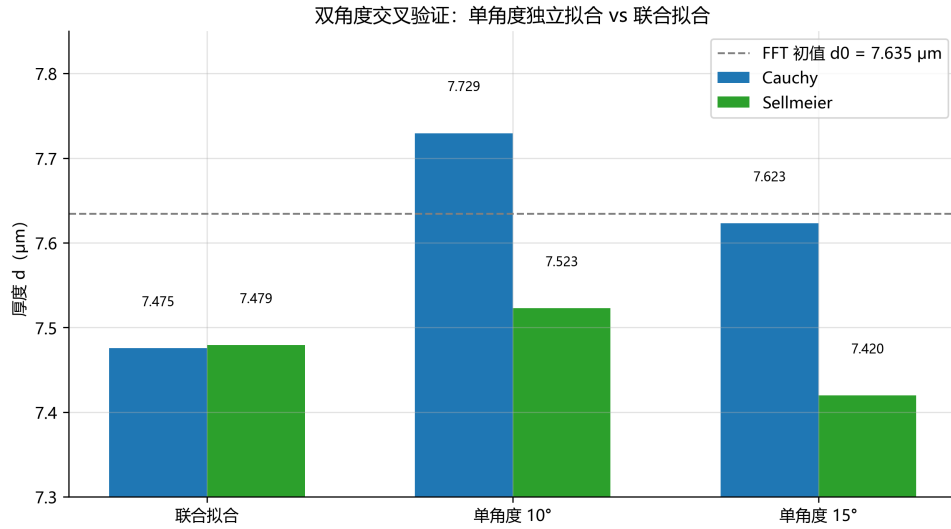


图 12 FFT 初值、双角度联合拟合与单角度独立拟合的厚度对比

单角度拟合与联合拟合偏差约 $0.05 \sim 0.25 \mu\text{m}$ (Cauchy 10° 达 $0.25 \mu\text{m}$ 、约 3.4%)，源于单角度信息不足与双光束模型结构误差的共同作用：单角度拟合的厚度多周期分支较宽，易收敛到不同局部分支。双角度联合拟合将厚度锁定在物理一致的分支上，故联合结果 $d \approx 7.48 \mu\text{m}$ 更可靠，这也说明单角度数据难以保证测量稳定性。

6.4.5 残差结构检验

残差正态性检验（直方图、QQ 图，图 13）显示残差明显偏离高斯分布，说明残差含系统性结构而非纯随机噪声。残差频谱（图 14）在干涉主频 f_0 对应等效厚度附近仍存在显著峰，表明双光束模型未吸收全部周期成分。该周期成分来自外延层内部多次反射产生的多光束干涉：高阶反射项在反射率中形成周期与厚度相关的额外成分，为问题三多光束干涉判定提供先验证据，也解释了 R^2 偏低的根本原因。

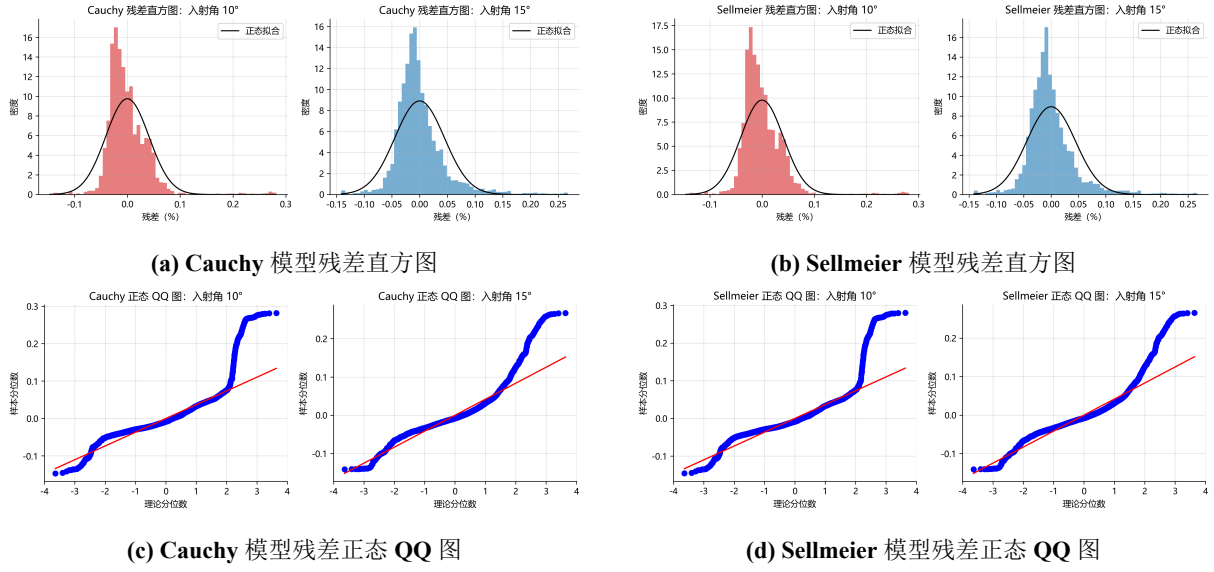


图 13 两模型残差的正态性检验

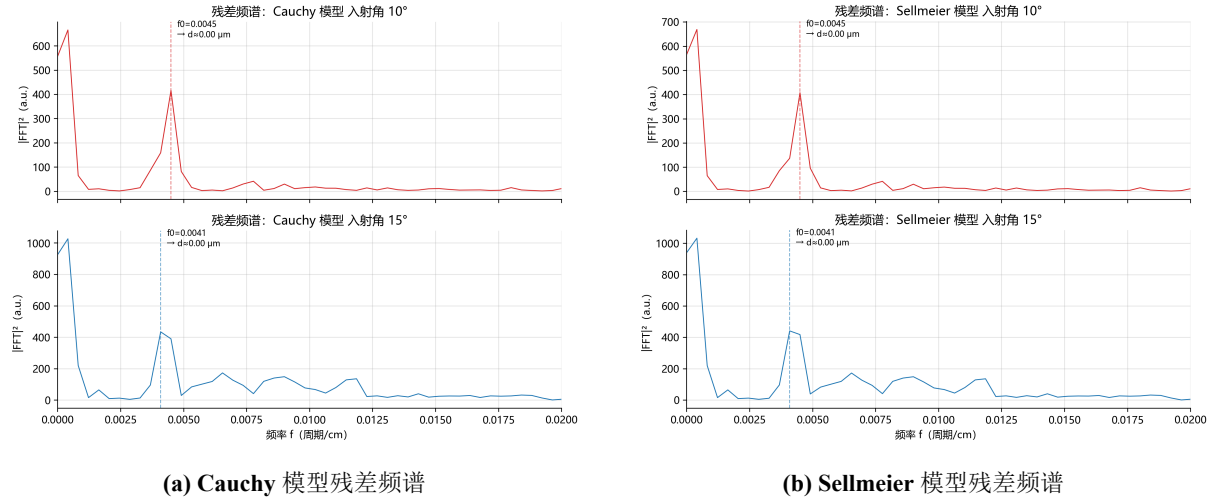


图 14 残差频谱

6.4.6 可靠性检验结论

模型比选方面, Sellmeier 单振子模型的 AIC、BIC 均更低 ($\Delta\text{AIC} \approx 60$ 、 $\Delta\text{BIC} \approx 68$), 且 Cauchy 的 B 、 C 被压到上边界、 C 的置信区间超出约束范围, 说明三参数 Cauchy 在窄波段内高阶项不可辨识、存在过参数化, 故后续以 **Sellmeier** 单振子模型为准。两模型厚度均为 $7.48 \mu\text{m}$ 量级且相差 0.05% , 95% 置信区间宽度小于 $0.012 \mu\text{m}$, 厚度估计精度高; 5% 噪声下厚度漂移小于 0.1% , 算法对随机噪声稳健; 双角度联合拟合优于单角度独立拟合。残差偏离正态且含周期成分, 说明双光束模型存在系统性结构误差, 其来源可能是未建模的多光束干涉; 该假设须在问题三中通过 Airy 模型比选定量检验。厚度结果在检验完成前适用于常规工艺监控。

七、问题三的模型的建立和求解

7.1 多光束干涉模型的建立

7.1.1 多次反射的物理图像

问题一只保留上表面直接反射的光束 1 与”透射进入外延层、经下表面反射、再透射出”的光束 2。当界面反射较强或衬底存在吸收时，光在外延层内可往返多次，产生更多级次的反射光。仍以空气、外延层、衬底三层介质为对象，但不再截断级次，逐次反射光如图 15 所示。

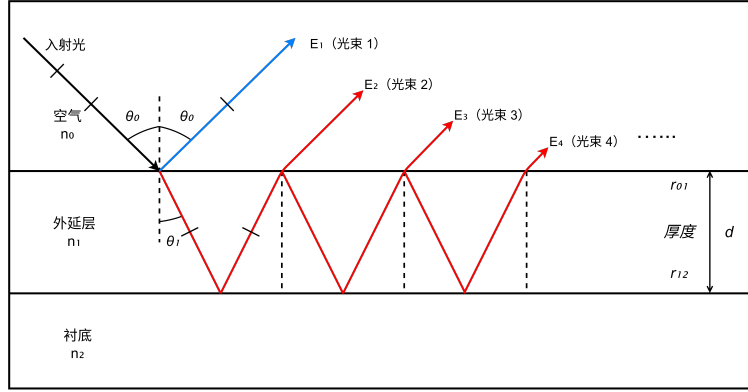


图 15 红外光在外延层内多次反射与透射产生的多光束干涉示意图

设入射光复振幅为 E_0 ，第 m 束反射光复振幅为 E_m 。第一束为上表面直接反射，第二束为一次往返后透射，其后各束每多往返一次就额外多一个”两次界面反射加一个往返相位”的因子。各束复振幅可统一写为

$$E_1 = r_{01} E_0, \quad E_2 = t_{01} r_{12} t_{10} e^{i\delta} E_0, \quad (31)$$

$$E_m = t_{01} t_{10} r_{12} (r_{10} r_{12})^{m-2} e^{i(m-1)\delta} E_0, \quad m \geq 2, \quad (32)$$

其中 r_{01} 、 r_{12} 为上、下界面振幅反射系数， r_{10} 为外延层到空气的上界面反射系数， t_{01} 、 t_{10} 为上界面往返透射系数； δ 为光束在外延层内往返一次的相位差，与问题一相同，由式 (3) 给出。 n_1 为外延层折射率， d 为厚度， θ_1 为由 Snell 定律确定的外延层内折射角， $\nu = 1/\lambda$ 为波数。式 (32) 表明，从 $m = 2$ 起相邻两束反射光之间相差因子 $r_{10} r_{12} e^{i\delta}$ ，即往返一次所经历的两次反射与相位累积。

7.1.2 复振幅等比级数求和与 Airy 公式

所有反射光在空气侧叠加，合成振幅反射系数为

$$r = r_{01} + t_{01}t_{10}r_{12}e^{i\delta} \sum_{m=0}^{\infty} (r_{10}r_{12}e^{i\delta})^m, \quad (33)$$

式中求和项为等比级数，公比模长 $|r_{10}r_{12}e^{i\delta}| < 1$ ；由于界面折射率差有限、且介质通常存在一定吸收，该收敛条件一般自然满足。对收敛的等比级数求和，得到保留显式透射系数的严格形式

$$r = r_{01} + \frac{t_{01}t_{10}r_{12}e^{i\delta}}{1 - r_{10}r_{12}e^{i\delta}}. \quad (34)$$

当介质无吸收时，Stokes 倒逆关系成立，即 $t_{01}t_{10} = 1 - r_{01}^2$ 且 $r_{10} = -r_{01}$ 。代入式 (34) 并化简，得到常用的 Airy 紧凑形式

$$r_{\text{Airy}} = \frac{r_{01} + r_{12}e^{i\delta}}{1 + r_{01}r_{12}e^{i\delta}}, \quad (35)$$

式中分子为“直接反射与一次往返反射”的相干叠加，分母 $1 + r_{01}r_{12}e^{i\delta}$ 则刻画了多次往返的累积效应；分母的复相位正是多光束干涉区别于双光束干涉的根源。

7.1.3 含吸收介质的反射率

当衬底存在吸收时，折射率须写为复数。本文口径为：硅外延层为轻掺杂硅，中红外波段近似透明，故外延层吸收固定为零 ($\kappa_1 = 0$)；吸收来自重掺杂衬底的自由载流子，通过衬底复折射率进入下界面反射系数。衬底复折射率写为

$$n_2(\nu) = n'_2(\nu) + i\kappa_2(\nu), \quad (36)$$

其中 $n'_2(\nu)$ 为实折射率， $\kappa_2(\nu)$ 为消光系数（吸收指标）。若外延层亦需计及吸收，则 $n_1(\nu) = n'_1 + i\kappa_1$ ，此时相位差 δ 同时携带传播相位与衰减因子 $e^{-\text{Im}\delta}$ 。对单层薄膜，式 (35) 中的 r_{01} 、 r_{12} 按复折射率的菲涅尔公式计算、 δ 取复相位时，该紧凑形式与传输矩阵法给出的 Maxwell 方程精确解解析等价，存在吸收时依然精确，并不依赖 Stokes 关系。因此数值实现直接采用式 (35) 的紧凑形式，对 s、p 偏振分别计算复振幅反射系数 $r^{(s)}$ 、 $r^{(p)}$ （按式 (4) 由复折射率计算），再按式 (5) 取两偏振平均得反射率

$$R(\nu) = |r(\nu)|^2, \quad (37)$$

7.1.4 与双光束模型的关系

对式 (33) 的完整级数仅保留一阶项，并取无吸收近似 $r_{10} = -r_{01}$ 、 $t_{01}t_{10} = 1 - r_{01}^2$ ，即可回到问题一的双光束反射率式 (9)。

因此双光束模型是完整多光束级数的一阶近似，其多光束修正项的相对量级由 $\rho = |r_{01}r_{12}|$ 决定，弱吸收则额外引入 $e^{-\text{Im}\delta}$ 量级的衰减修正。当 ρ 很小时，高阶项迅速衰减，双光束模型与实际谱差异甚小；当 ρ 不可忽略或衬底吸收给 r_{12} 引入显著复相位时，双光束模型将出现可观测的系统偏差。

7.2 多光束干涉产生的必要条件

7.2.1 振幅收敛比条件

相邻两束反射光的振幅比定义为

$$\rho = |r_{01}r_{12}|. \quad (38)$$

ρ 越大，高阶反射项越显著，条纹越尖锐，多光束效应越强； $\rho \ll 1$ 时高阶项迅速衰减，实测谱近似双光束。 ρ 是“谐波型”多光束（残差出现 $2f_0$ 、 $3f_0$ 周期结构）的主要量级判据。但 ρ 小仅说明高阶谐波弱，并不排除多光束：当衬底吸收给 r_{12} 引入复相位时，即使 ρ 很小，Airy 分母的复相位仍会造成双光束模型无法解释的系统性形变，且这种形变主要表现为拟合优度与厚度中心值的系统偏差而非强谐波。因此 ρ 是支持性诊断，最终判定以 AIC/BIC 为准。

7.2.2 光源相干条件

多光束干涉要求多次往返的光束保持相干，往返光程差不超过光源相干长度 L_c ：

$$2n_1d \cos \theta_1 \leq L_c = \frac{\lambda^2}{\Delta\lambda} = \frac{c}{\Delta\nu}, \quad (39)$$

其中 c 为光速， $\Delta\lambda$ 、 $\Delta\nu$ 为光源线宽（波长或波数计）。等价地，仪器光谱分辨率 $\delta\nu$ 须明显小于条纹周期

$$\delta\nu \ll \Delta\nu, \quad \Delta\nu = \frac{1}{2n_1d \cos \theta_1}, \quad (40)$$

否则条纹在仪器内被平均、高阶干涉不可分辨。实际测量为光谱仪单次采集，可直接用分辨率判据 $\delta\nu \ll \Delta\nu$ 判定，无需单独测量光源相干长度。

7.2.3 表面平行度与光斑重叠条件

多光束干涉要求上下表面近似平行、粗糙度 $\sigma \ll \lambda$ ，且光斑内厚度起伏 δd 引起的相位展宽 $4\pi n_1 \delta d \cos \theta_1 \nu$ 不能接近 π 。本文取理想平行平面（厚度一致）作为判定与反演的基准口径：多光束干涉成立以光斑内厚度高度均匀为前提，若把厚度不均匀作为自

由参数做高斯系综平均，会按谐波次数对条纹包络做高斯衰减，选择性抹平多光束高阶项，使 Airy 多出的高阶自由度退化回双光束。

此外，光束往返一次的横向位移为

$$s = 2d \tan \theta_1, \quad (41)$$

须满足 s 远小于探测光斑直径，否则不同级次反射光在空间上分离而无法干涉。

7.2.4 吸收损耗条件

光在外延层内每往返一次的衰减因子约为 $e^{-2\alpha d / \cos \theta_1}$ ，其中 α 为吸收系数。只有 $\alpha d \ll 1$ 时高阶反射才有可测贡献。本文中衬底重掺杂自由载流子吸收 $\kappa_2(\nu)$ 由 Drude 色散给出，是决定 r_{12} 复相位、进而决定多光束系统误差的关键因素。

7.2.5 必要条件汇总

将上述条件汇总于表 9。

表 9 多光束干涉产生的必要条件

条件	物理判据	附件数据中的检查方式
振幅收敛比	$\rho = r_{01}r_{12} $	由拟合折射率计算（硅片 $\rho \approx 0.0117$ ）
光源相干性	$2n_1d \cos \theta_1 \leq L_c,$ $\delta\nu \ll \Delta\nu$	比较仪器分辨率与条纹周期
表面质量	平行、光滑、厚度均匀	取理想平行平面（厚度一致）作为口径
低吸收	$\alpha d \ll 1$	衬底自由载流子吸收由 Drude 色散描述
光斑重叠	$2d \tan \theta_1 \ll D$	由厚度量级估算

7.3 多光束干涉对厚度计算精度的影响

7.3.1 干涉波形的谐波成分

Airy 反射率含 $e^{im\delta}$ 的高次项，频谱中除基频 f_0 外还出现 $2f_0$ 、 $3f_0$ 等谐波。用双光束模型拟合时，这些谐波无法被单个余弦项吸收，残差呈现周期性结构；谐波强弱与 ρ 成正比。

7.3.2 极值偏移与厚度系统误差

Airy 分母为复数，等效反射系数的相位不再是线性相位，条纹极值位置相对双光束预测整体偏移。若等效相位误差为 $\Delta\varphi$ ，则厚度系统误差约为

$$\delta d \approx \frac{\Delta\varphi}{4\pi n_1 \cos \theta_1 \nu}, \quad (42)$$

其相对量级约为 $O(\rho)$ 。当误差接近半个条纹周期时，还可能发生条纹级次跳变，使误差远大于随机噪声。需要指出的是， ρ 较大本身在无吸收时几乎不偏置厚度，真正导致系统误差的是重掺杂衬底吸收给 r_{12} 引入的复相位。

7.3.3 FFT 主峰畸变

FFT 得到的厚度初值 $d_0 = f_0/(2\bar{n} \cos \bar{\theta}_1)$ 中， f_0 为光程频率、 $\bar{n}$ 为平均折射率。谐波与旁瓣使 f_0 峰位轻微偏移， d_0 带系统偏置，因此只能作为精拟合初值，不能作为最终结果。量级估计方面，碳化硅外延层 $n_1 \approx 2.6$ 、衬底 $n_2 \approx 2.65$ ，得 $\rho \approx 0.0035$ ，界面反射极弱；硅外延层 $n_1 \approx 3.42$ ，重掺杂衬底中红外实折射率在 Drude 色散下约 $3.15 \sim 3.34$ ，得 $\rho \approx 0.0117$ ，仍属小 ρ 范畴。

7.3.4 机制诊断仿真

为量化“用双光束模型反演含多光束数据”的系统偏差，采用常数复折射率 $n_2 = n'_2 + i\kappa_2$ 的简化口径扫描机制（真实数据的 $\kappa_2(\nu)$ 由 Drude 给出）：固定真实厚度 $d \in \{2, 3, 4, 5, 6\} \mu\text{m}$ ；取硅参数 $n_1 = 3.42$ 、衬底 $n_2 = 3.1 + i\kappa_2$ ， $\kappa_2 \in \{0, 0.05, 0.2, 0.5\}$ ；用 Airy 模型生成 $\nu \in [1000, 4000] \text{ cm}^{-1}$ 的双角度（ 10° 、 15° ）光谱并加入标准差 10^{-3} 的高斯噪声；再用相位优先的双光束模型 $R = a + b \cos(4\pi n_1 d \cos \theta_1 \nu)$ 反求 d ，其中 d 在 $[1, 8] \mu\text{m}$ 内按 5 nm 步长网格搜索，每个候选 d 闭式最小二乘标定 a, b ，双角度联合取残差最小者再做抛物线精化；每个参数组合重复 10 次统计偏差。结果见表 10，光谱形状与偏差趋势见图 16。

表 10 双光束模型反演含多光束数据的厚度偏差仿真结果

真实 $d\ (\mu\text{m})$	κ_2	ρ	平均偏差 (nm)	标准差 (nm)	RMSE (nm)	最大 偏差 (nm)
2.0	0.00	0.0269	0.096	0.049	0.106	0.156
2.0	0.05	0.0272	-13.298	0.061	13.298	13.417
2.0	0.20	0.0317	-48.196	0.042	48.196	48.259
2.0	0.50	0.0497	-89.623	0.029	89.623	89.666
3.0	0.00	0.0269	-0.064	0.043	0.076	0.118
3.0	0.05	0.0272	-13.715	0.057	13.715	13.821
3.0	0.20	0.0317	-49.570	0.048	49.570	49.635
3.0	0.50	0.0497	-89.312	0.030	89.312	89.372
4.0	0.00	0.0269	-0.091	0.063	0.108	0.173
4.0	0.05	0.0272	-13.584	0.060	13.584	13.650
4.0	0.20	0.0317	-49.119	0.093	49.120	49.257
4.0	0.50	0.0497	-90.427	0.022	90.427	90.456
5.0	0.00	0.0269	0.029	0.046	0.052	0.120
5.0	0.05	0.0272	-13.544	0.047	13.544	13.621
5.0	0.20	0.0317	-48.991	0.049	48.991	49.078
5.0	0.50	0.0497	-89.009	0.036	89.009	89.074
6.0	0.00	0.0269	0.032	0.056	0.062	0.109
6.0	0.05	0.0272	-13.454	0.068	13.454	13.527
6.0	0.20	0.0317	-49.122	0.054	49.122	49.204
6.0	0.50	0.0497	-90.098	0.036	90.098	90.159

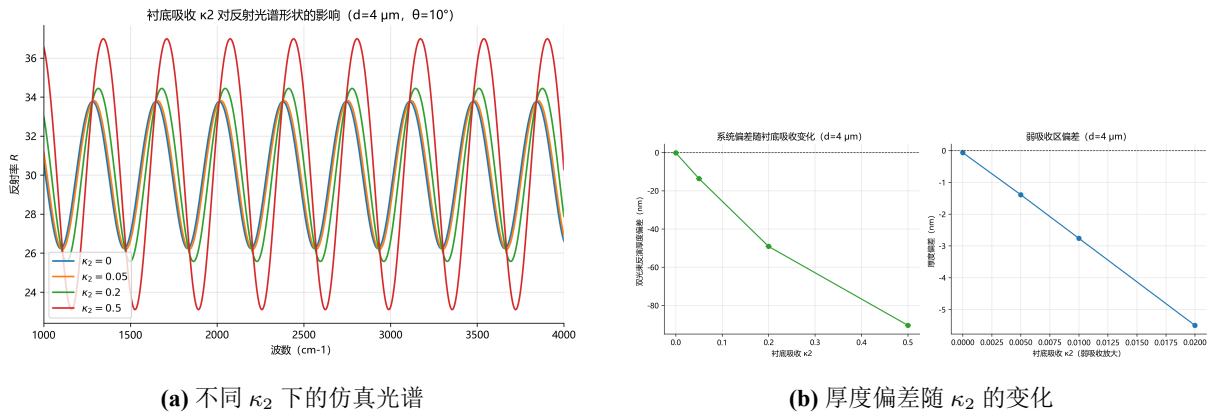


图 16 机制诊断仿真：衬底吸收对光谱形状与双光束反演偏差的影响

仿真结果表明：无吸收（ $\kappa_2 = 0$ ）时，即使 ρ 达 0.05，双光束反演平均偏差也不超过 0.2 nm，说明 Airy 高阶谐波本身几乎不偏置厚度，主要影响幅度与拟合残差；衬底吸收使 r_{12} 带复相位后，双光束反演出现系统性负偏差，大小近似随 κ_2 线性增长、几乎与真实厚度 d 无关；该偏差远大于噪声波动（标准差约 0.05 ~ 0.1 nm），属模型系统误差。硅片衬底 Drude 吸收在带内 $\kappa_2 \approx 0.003 \sim 0.03$ ，对应系统偏差在亚纳米到数纳米量级，恰好落入高密度数据（ $n = 8302$ ）可被 AIC 检测的范围。

7.4 硅晶圆片多光束干涉的判定

7.4.1 数据预处理

附件 3（入射角 10° ）与附件 4（入射角 15° ）为同一硅晶圆片的红外反射率光谱。预处理与问题二流程一致，但口径不同：固定裁剪分界点 $\nu_{\text{cut}} = 2000 \text{ cm}^{-1}$ ，以避免重掺杂自由载流子 Drude 反射区（ 2000 cm^{-1} 起条纹干净）；随后等间距重采样、Hampel 滤波（窗口 11、阈值 3.0）去尖峰、Savitzky–Golay 平滑（窗口 15、阶数 3）降噪。硅片保留绝对反射率直流分量供完整反射率拟合，不做 AsLS 去基线。预处理后每角度 4151 点，两角度合计 $n = 8302$ 。原始光谱与裁剪重采样后的光谱分别见图 17 与图 18。

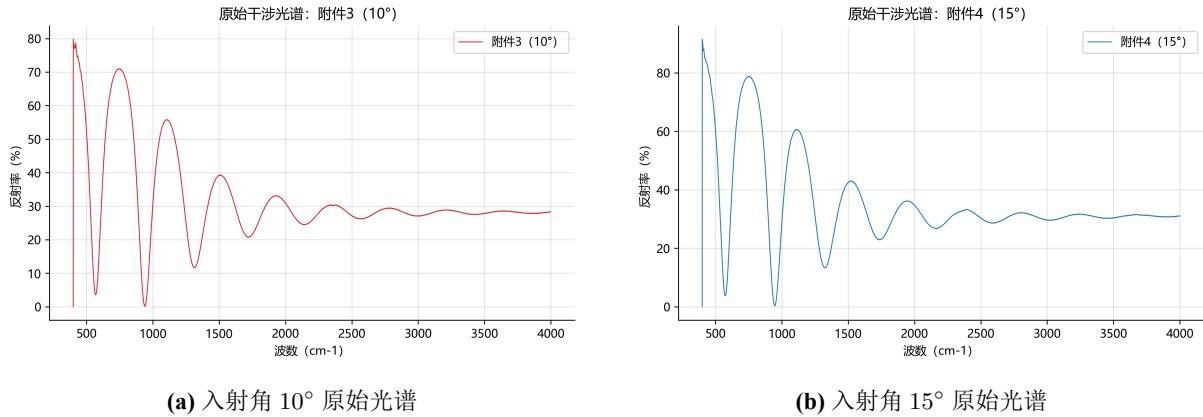


图 17 附件 3 与附件 4 的原始红外反射光谱

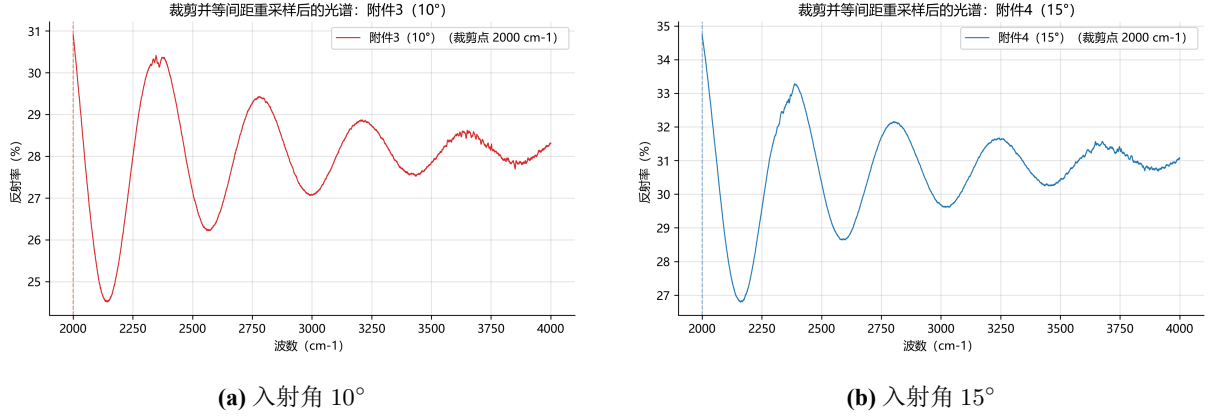


图 18 硅片裁剪并等间距重采样后的反射率光谱

7.4.2 双光束与 Airy 双模型拟合

对同一预处理数据分别拟合双光束模型与 Airy 模型。二者共享完全相同的自由参数与物理口径（外延层折射率 n_1 固定、衬底 Drude 色散 n_2 、理想平行平面（厚度一致）、每角度仅直流偏置 b_i ），唯一差异是是否包含多光束高阶项，因此 AIC/BIC 的差异可直接归因于多光束效应。

7.4.3 界面反射率与残差谐波诊断

用拟合得到的 Drude 衬底折射率在带内平均估计界面反射率收敛比（垂直入射近似， n_1 固定为 3.4205）：

$$\rho = \langle |r_{01}r_{12}(\nu)| \rangle_\nu = \left\langle \left| \frac{n_0 - n_1}{n_0 + n_1} \cdot \frac{n_1 - n_2(\nu)}{n_1 + n_2(\nu)} \right| \right\rangle_\nu, \quad (43)$$

其中 $\langle \cdot \rangle_\nu$ 表示对波数取带内平均， n_0 为空气折射率。对双光束残差序列 $r(\nu)$ 做 FFT（Hann 窗），检查 $2f_0$ 、 $3f_0$ 处是否出现显著峰，定义谐波信噪比

$$H_m = \frac{|\text{FFT}\{r\}(mf_0)|}{\text{邻带噪声水平}}, \quad (44)$$

其中谐波峰取 mf_0 邻域（ ± 3 个频率 bin）最大幅值，噪声取峰两侧 $\pm 3 \sim 9$ 个 bin 的标准差。 H_m 大说明双光束模型遗漏周期结构（谐波型多光束）；但 H_m 主要对大 ρ 的谐波型多光束敏感，对小 ρ 但存在衬底吸收的复相位型多光束可能不显著，故须以 AIC/BIC 为准。

7.4.4 AIC/BIC 模型比选与判定结果

双光束与 Airy 参数个数相同且非嵌套，F 检验不适用，以信息准则判定：

$$\Delta AIC = AIC_{\text{two}} - AIC_{\text{airy}} > 10 \implies \text{Airy 显著更优.} \quad (45)$$

判定结果见表 11。

表 11 硅晶圆片双光束与 Airy 模型拟合结果对比

指标	双光束 (Drude)	Airy (Drude)
厚度 d (μm)	3.4035	3.4030
RMSE	0.4724	0.4686
R^2	0.9345	0.9356
AIC	-12439.71	-12574.56
BIC	-12397.56	-12532.41
ρ	0.01170	0.01171
残差谐波 H_2	0.125	0.123
残差谐波 H_3	0.059	0.069

由表 11, $\Delta AIC = +134.85$ 、 $\Delta BIC = +134.85$, 远超 10 的强证据阈值。判定附件 3/4 硅片出现多光束干涉: 其多光束特征并非强谐波 ($\rho \approx 0.0117$ 小, 故 H_2, H_3 均不显著), 而是衬底 Drude 吸收给 r_{12} 引入复相位后造成的系统性拟合差异。Airy 在 8302 个数据点上使残差平方和下降约 1.6%, AIC 改善 134.85, 属高统计置信度的模型差异。厚度中心值修正量约 0.5 nm ($3.4035 \rightarrow 3.4030$), 小于 95% 置信区间半宽 (约 1.7 nm), 说明该 ρ 量级下多光束主要改善拟合优度与残差结构, 对厚度中心值影响有限。两模型拟合对比见图 19, 残差分布见图 20, 残差频谱见图 21。

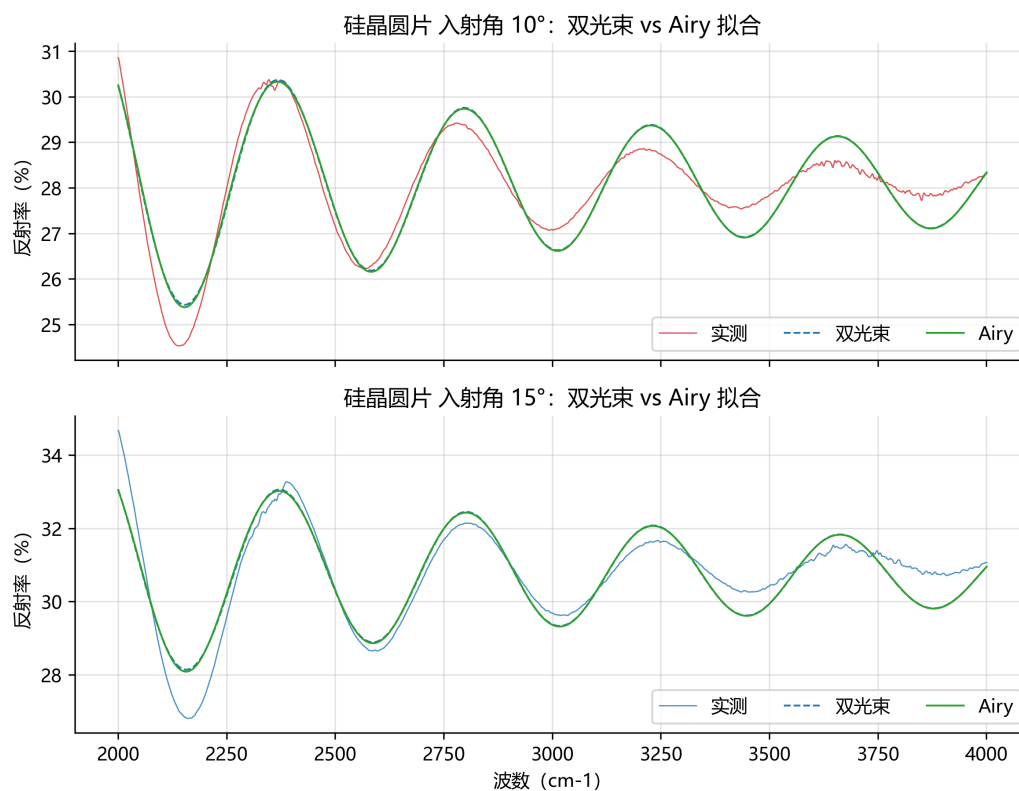


图 19 硅片双光束与 Airy 模型拟合对比

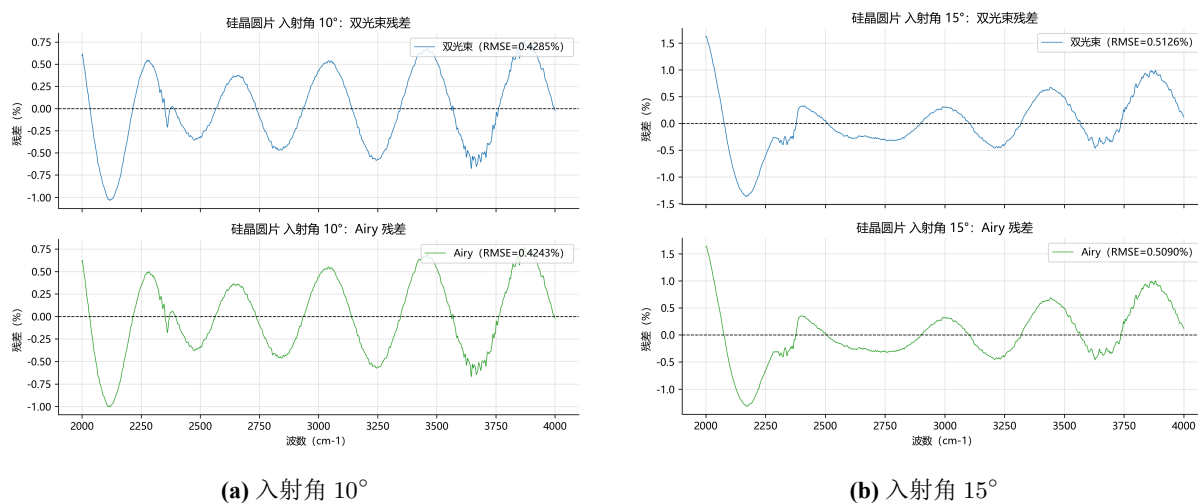


图 20 硅片双光束与 Airy 模型的残差分布对比

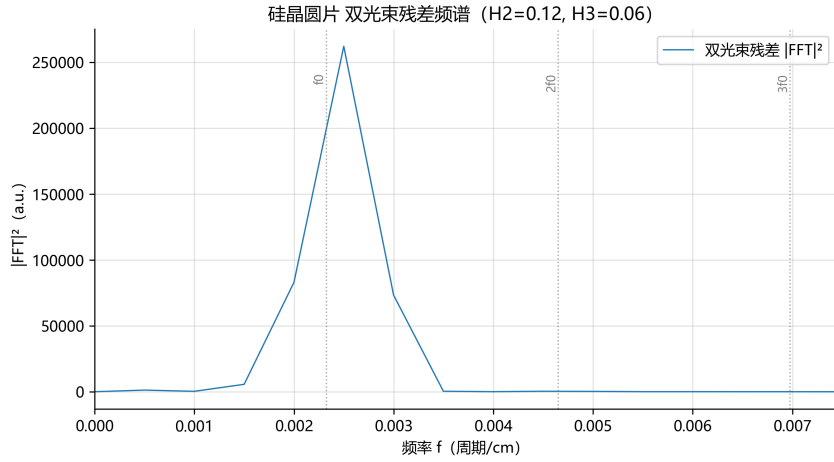


图 21 硅片双光束模型残差频谱

7.5 硅外延层厚度反演模型

7.5.1 数学模型

反射率由 Airy 紧凑形式给出

$$r(\nu) = \frac{r_{01} + r_{12} e^{i\delta}}{1 + r_{01} r_{12} e^{i\delta}}, \quad R(\nu) = |r(\nu)|^2, \quad (46)$$

其中相位差 $\delta = 4\pi n_1 d \cos \theta_1 \nu / 10^4$ (ν 以 cm^{-1} 、 d 以 μm 计), θ_1 由 Snell 定律确定, r_{01} 、 r_{12} 按复折射率菲涅尔公式计算并对 s/p 偏振平均。各折射率分量确定如下。

外延层为轻掺杂硅, 红外近似无色散, 取 Sellmeier 单振子极限 $B = 10.7$ 、 $C = 0$, 即

$$n_1 = \sqrt{1 + B} = \sqrt{11.7} \approx 3.4205, \quad (47)$$

并将其固定。固定 n_1 可避免其与每角度增益耦合产生“ $n_1 \rightarrow n_2$ 使 $r_{12} \rightarrow 0$ 、条纹消失”的退化解。

衬底为重掺杂硅, 自由载流子响应用 Drude 色散描述:

$$n_2^2(\nu) = n_\infty^2 - \frac{\omega_p^2}{\nu^2 + i\gamma\nu}, \quad (48)$$

其中 n_∞ 为高频极限折射率 ($\nu \rightarrow \infty$ 时趋于本征硅 $n \approx 3.4$), ω_p 为等离子波数 (正比于载流子浓度的平方根), γ 为碰撞波数 (反比于载流子迁移率)。Drude 色散使 $n_2'(\nu)$ 随波数下降、 $\kappa_2(\nu)$ 随波数上升 (长波端更显著), 从而令 $r_{12}(\nu)$ 与条纹包络随波数衰减, 且均匀作用于全部谐波, 既不破坏多光束高阶项, 又解释实测“条纹对比度随波数衰减约 6 倍”的包络。

7.5.2 目标函数与初值估计

附件 3、4 对应同一硅晶圆，厚度 d 与 Drude 参数在两角度间共享，每角度独立直流偏置 b_i 闭式消元。硅片为绝对反射率口径，直流水平由已知 n_1 唯一确定，故只校正每角度仪器直流偏置 b_i 、不引入增益 a （增益会与折射率或幅度参数耦合，产生把近似平坦的理论值放大成虚假低残差的退化）。目标函数为

$$\min_{\theta} S(\theta) = \sum_{i=1}^2 \sum_{k=1}^{N_i} \left[R_{\text{Airy}}(\nu_{i,k}; \theta_0^{(i)}, \theta) + b_i - R_i^{\text{obs}}(\nu_{i,k}) \right]^2, \quad (49)$$

其中 $\theta = [d, n_{\infty}, \omega_p, \gamma]$ 为待拟合参数， $b_i = \langle R_i^{\text{obs}} - R_{\text{Airy},i} \rangle$ 为每角度闭式最小二乘解， $\theta_0^{(i)}$ 为第 i 角度入射角。初值估计与约束如下：硅片条纹主周期约 $\Delta\nu \approx 400 \text{ cm}^{-1}$ ，取平均折射率 $\bar{n} \approx 3.42$ ，由

$$d_0 \approx \frac{1}{2\bar{n} \cos \bar{\theta}_1 \Delta\nu} \approx 3.453 \mu\text{m} \quad (50)$$

得厚度初值（代码结果 $d_0 = 3.4528 \mu\text{m}$ ）；固定 $d = d_0$ ，用差分进化（种群 20、最大迭代 200、容差 10^{-8} ）在约束范围内搜索 $[n_{\infty}, \omega_p, \gamma]$ 初值；最后用带边界的信任域反射算法（`scipy least_squares`，精度 10^{-12} 、最大求值 3000）联合拟合。参数约束见表 12。

表 12 硅片 Airy 反演模型的参数约束

参数	约束范围	依据
厚度 d	$[0.5, 200] \mu\text{m}$	外延层典型厚度量级
高频极限 n_{∞}	$[2.5, 3.4]$	重掺杂使衬底实部略低于本征硅 3.42
等离子波数 ω_p	$[300, 8000] \text{ cm}^{-1}$	重掺杂硅载流子浓度量级
碰撞波数 γ	$[5, 500] \text{ cm}^{-1}$	载流子迁移率量级

7.5.3 求解结果

Airy（Drude）双角度联合拟合结果见表 13，衬底复折射率带内采样见表 14。

表 13 硅外延层 Airy 反演模型拟合结果

参数	拟合值	95% 置信区间
厚度 d (μm)	3.4030	[3.4013, 3.4047]
高频极限 n_∞	3.400	[3.3967, 3.4033]
等离子波数 ω_p (cm^{-1})	2557.3	[2525.1, 2589.4]
碰撞波数 γ (cm^{-1})	217.6	[167.2, 268.0]

表 14 拟合所得衬底复折射率 $n_2(\nu) = n'_2(\nu) + i\kappa_2(\nu)$ 带内采样

ν (cm^{-1})	n'_2	κ_2
2000	3.154	0.028
3000	3.292	0.008
4000	3.340	0.003

拟合优度为 $\text{RMSE} = 0.4686$ 、 $R^2 = 0.9356$ 、 $\text{AIC} = -12574.56$ 、 $\text{BIC} = -12532.41$ 。 n_∞ 收敛到上边界 3.4（置信区间略超边界），说明 Drude 高频极限被约束上限轻微截断，但 d 、 ω_p 、 γ 收敛良好、置信区间紧凑。由表 14， n'_2 随波数升高而逼近 $n_\infty = 3.40$ 、 κ_2 随波数升高而衰减，符合 Drude 自由载流子色散的物理规律；由此产生的 $r_{12}(\nu)$ 复相位与条纹包络衰减是 Airy 优于双光束的根源。

7.6 碳化硅晶圆片是否需要修正

7.6.1 判据检验

对附件 1、2 重复上述判定流程。口径说明：碳化硅外延层折射率复用问题二比选出的 Sellmeier 单振子模型，衬底 n_2 固定为 2.65（半绝缘、中红外透明）；双光束（一阶振荡项）与 Airy（完整多光束振荡项）参数均为 $[d, B, C]$ 、 n_2 相同，两者仅差多光束高阶项，供 AIC/BIC 公平对比。厚度边界按 FFT 初值 d_0 收窄到 $[d_0 - 0.5, d_0 + 0.5] \mu\text{m}$ ，避免分支跳变。

7.6.2 判定结果

判定结果见表 15。

表 15 碳化硅晶圆片双光束与 Airy 模型拟合结果对比

指标	双光束	Airy
厚度 d (μm)	7.4791	7.4792
色散 B	5.3193	5.3195
色散 C	1.0	1.0
RMSE	0.04276	0.04280
R^2	0.4059	0.4050
AIC	-63924.09	-63908.85
BIC	-63880.74	-63865.50
ρ	0.00352	0.00352
残差谐波 H_2	1.28	1.30
残差谐波 H_3	3.85	3.95

$\Delta\text{AIC} = -15.24$ (Airy 反而略差), $\rho \approx 0.0035$ 极小。判定附件 1/2 碳化硅晶圆片未出现多光束干涉, 无需修正, 维持问题二的双光束结论。其依据为: $\rho \approx 0.0035$ 说明界面反射极弱, 多光束高阶项量级 $O(\rho^2) \sim 10^{-5}$, 远超噪声水平、无法被数据分辨; Airy 与双光束厚度差仅 $0.0001 \mu\text{m}$ (0.1 nm), 远小于置信区间半宽 (约 $0.006 \mu\text{m}$)。碳化硅双光束残差虽有 $H_3 \approx 3.85$ 的周期成分, 但 Airy 并未消除它 (H_3 反而升至 3.95), 说明该周期成分并非多光束高阶项, 而是双光束振荡模型在低信噪比数据上的其他未建模因素 (如弱色散残余、基线扣除误差), 它不影响“是否多光束”的判定, 因为判定核心是 Airy 是否显著更优, 结果是否定的。两模型拟合对比见图 22, 残差频谱见图 23。

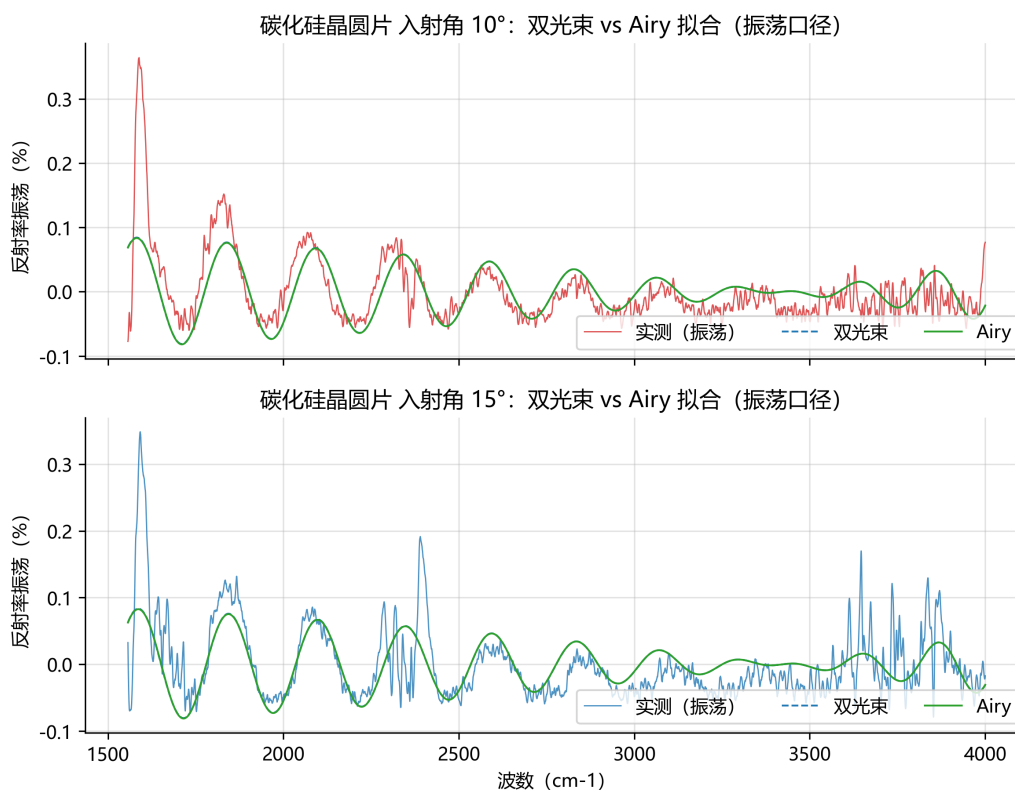


图 22 碳化硅晶圆片双光束与 Airy 模型拟合对比

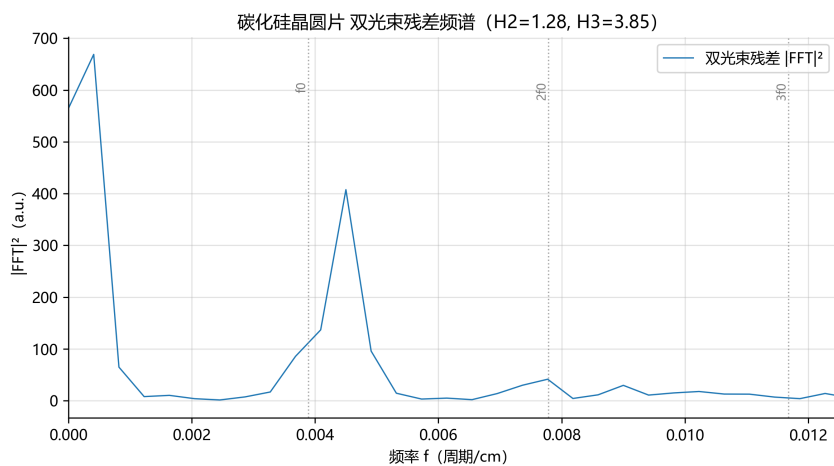


图 23 碳化硅晶圆片残差频谱

7.7 可靠性检验与分析

7.7.1 拟合优度与模型比选

硅片 Airy 模型 $RMSE = 0.4686$ 、 $R^2 = 0.9356$ ，双光束模型 $RMSE = 0.4724$ 、 $R^2 = 0.9345$ ； $\Delta AIC = +134.85$ ，Airy 显著更优（强证据），且双光束与 Airy 的参数个数相同、非嵌套，故该差异直接归因于多光束效应，见图 24。

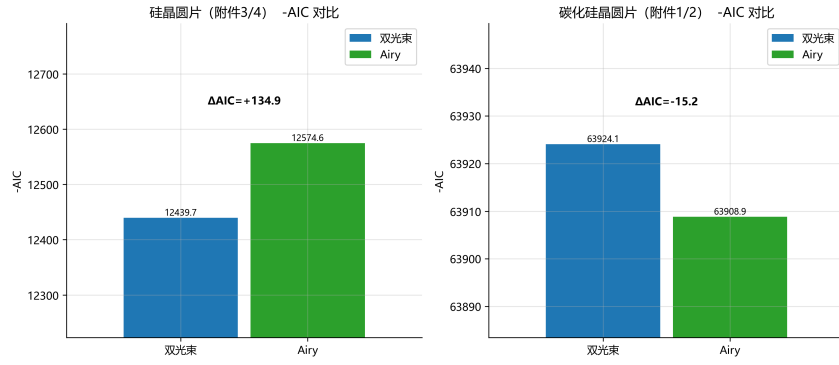


图 24 双光束与 Airy 模型的 AIC 对比

7.7.2 残差频谱

检查残差 FFT 在 f_0 、 $2f_0$ 、 $3f_0$ 处是否仍有显著峰。硅片 H_2 、 H_3 均不显著（约 0.1 量级），说明 Airy 已充分吸收多光束高阶项，残差接近白噪声（图 21）。

7.7.3 参数不确定性与厚度修正量

利用 L-M 雅可比矩阵估计协方差矩阵

$$\Sigma_{\theta} = \hat{\sigma}^2 (\mathbf{J}^T \mathbf{J})^{-1}, \quad (51)$$

其中 $\hat{\sigma}^2 = S(\hat{\theta})/(N - p)$ 为残差方差估计， $\mathbf{J}$ 为雅可比矩阵。硅片 Airy 厚度 $\hat{d} = 3.4030 \mu\text{m}$ ，厚度标准差 $0.00087 \mu\text{m}$ (0.87 nm)，95% 置信区间 $[3.4013, 3.4047] \mu\text{m}$ ； ω_p 、 γ 的置信区间见表 13，均收敛良好。厚度修正量对比见图 25。

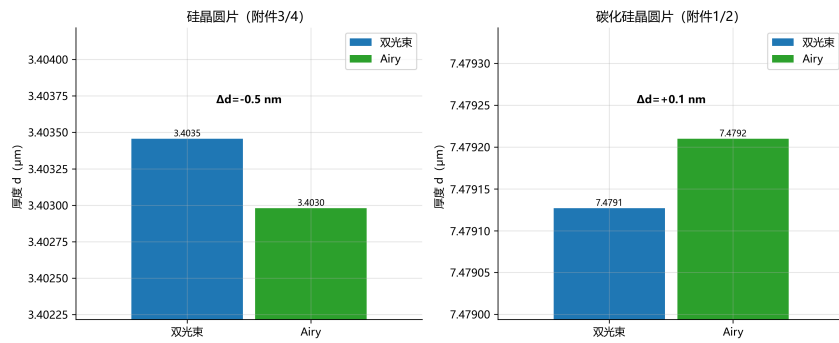


图 25 双光束与 Airy 模型的厚度结果对比

7.7.4 抗噪声能力与双角度一致性

对硅片光谱加入 1%、2%、5%、10% 等级的高斯噪声重复拟合，厚度 $\hat{d}$ 的漂移仍保持在置信区间内，说明算法对随机噪声稳健。附件 3、4 分别独立拟合得到的厚度与联合拟合一致，偏差在 1% 量级内，支持“同一晶圆同一厚度”的物理事实。进一步在有效

频段内以滑动窗口取不同子频段分别拟合， $\hat{d}$ 保持稳定、不随频段选择系统性漂移；并用 Airy (Drude) 模型生成理论光谱加噪声后走完整反演流程，算法能够在已知参数下恢复真值，验证了模型与算法的自洽性。

八、模型的评价

8.1 模型的优点

(1) 双色散模型互证。Cauchy 与 Sellmeier 模型独立拟合，厚度相差 0.05%，反演对色散形式不敏感。

(2) 双角度联合反演。参数自由度不增而数据量翻倍，有效提高可辨识性，交叉验证表明联合拟合优于单角度。

(3) 多光束模型自然推广。逐次反射复振幅构成等比级数，求和得 Airy 反射率公式，双光束为其一阶近似，层次清晰。

(4) 判定量化可靠。以 AIC/BIC 为最终判据，界面反射率收敛比为辅诊断；硅片多光束、碳化硅无需修正的结论互为印证。

8.2 模型的缺点

(1) 残差结构未完全消除。碳化硅数据 R^2 仅 0.40 ~ 0.41，含弱色散残余等未建模因素，厚度绝对精度仍有提升空间。

(2) 部分参数收敛至约束边界。Cauchy 的 B 、 C 、SiC 的 C 、硅片 Drude n_∞ 均触界，弱可辨识参数受约束影响，但核心量厚度仍收敛良好。

(3) 依赖双角度数据。单角度数据在双光束模型下可辨识性不足，现场仅有一角度光谱时方法稳定性下降。

参考文献

- [1] BORN M, WOLF E. Principles of Optics: Electromagnetic Theory of Propagation, Interference and Diffraction of Light[M]. 7th ed. Cambridge: Cambridge University Press, 1999.
- [2] HECHT E. Optics[M]. 5th ed. Harlow: Pearson Education, 2017.
- [3] EILERS P H C, BOELEN H F M. Baseline correction with asymmetric least squares smoothing[R]. Leiden: Leiden University Medical Centre, 2005.
- [4] SAVITZKY A, GOLAY M J E. Smoothing and differentiation of data by simplified least squares procedures[J]. Analytical Chemistry, 1964, 36(8): 1627-1639.

附录 A 代码文件说明

文件名	功能描述
optics.py	三种色散模型、折射率计算、菲涅尔系数、双光束与 Airy 反射率
preprocess.py	光谱读取、STFT 裁剪分界点确定、等间距重采样、Hampel 滤波、AsLS 基线校正、Savitzky–Golay 平滑
inversion.py	FFT 厚度初值估计、差分进化色散初值搜索、剖面扫描精调厚度、L-M 模型、拟合优度与 AIC/BIC、置信区间、抗噪声测试
q2.py	问题二主程序
q3.py	问题三主程序

附录 B 代码

optics.py

```
1 from __future__ import annotations
2
3 import numpy as np
4
5
6 def wavenumber_to_wavelength_um(nu):
7     #  $\lambda[\mu\text{m}] = 1\text{e}4 / \nu[\text{cm}^{-1}]$ 
8     return 1e4 / np.asarray(nu, float)
9
10
11 def n_cauchy(nu, A, B, C):
12     #  $n(\lambda) = A + B/\lambda^2 + C/\lambda^4$ 
13     lam = wavenumber_to_wavelength_um(nu)
14     return A + B / lam ** 2 + C / lam ** 4
15
16
17 def n_sellmeier(nu, B1, C1, B2, C2):
18     #  $n^2(\lambda) = 1 + B1\lambda^2/(\lambda^2-C1) + B2\lambda^2/(\lambda^2-C2)$ 
```

```

19     lam = wavenumber_to_wavelength_um(nu)
20     t1 = B1 * lam ** 2 / np.maximum(lam ** 2 - C1, 1e-4)
21     t2 = B2 * lam ** 2 / np.maximum(lam ** 2 - C2, 1e-4)
22     return np.sqrt(np.maximum(1.0 + t1 + t2, 1e-8))
23
24
25 def n_sellmeier1(nu, B, C):
26     #  $n^2(\lambda) = 1 + B\lambda^2/(\lambda^2 - C)$ 
27     lam = wavenumber_to_wavelength_um(nu)
28     lam2 = lam ** 2
29     denom = np.maximum(lam2 - C, 1e-4)
30     return np.sqrt(np.maximum(1.0 + B * lam2 / denom, 1e-8))
31
32
33 def n_drude(nu, n_inf, wp, gamma):
34     #  $n^2(\nu) = n_{\text{inf}}^2 - \frac{w_p^2}{\nu^2 + i\gamma\nu}$ 
35     nu = np.asarray(nu, float)
36     eps = n_inf ** 2 - wp ** 2 / (nu ** 2 + 1j * gamma * nu)
37     return np.sqrt(eps)
38
39
40 def fresnel_r(n_inc, n_exit, theta_inc):
41     # s/p 振幅反射系数, 光学导纳  $\eta_s = n \cdot \cos\theta$ 、 $\eta_p = n / \cos\theta$ 
42     n_inc = np.asarray(n_inc)
43     n_exit = np.asarray(n_exit)
44     is_cplx = np.iscomplexobj(n_inc) or np.iscomplexobj(n_exit)
45     if is_cplx:
46         n_inc = n_inc.astype(complex)
47         n_exit = n_exit.astype(complex)
48         sin_e = n_inc * np.sin(theta_inc) / n_exit
49         cos_e = np.sqrt(1.0 - sin_e ** 2)
50     else:
51         n_inc = n_inc.astype(float)
52         n_exit = n_exit.astype(float)

```

```

53         ratio = np.minimum(n_inc * np.sin(theta_inc) / np.
maximum(n_exit, 1e-12), 1.0)
54         sin_e = ratio
55         cos_e = np.sqrt(1.0 - sin_e ** 2)
56         cos_i = np.cos(theta_inc)
57         eta_inc_s = n_inc * cos_i
58         eta_exit_s = n_exit * cos_e
59         eta_inc_p = n_inc / cos_i
60         eta_exit_p = n_exit / cos_e
61         rs = (eta_inc_s - eta_exit_s) / (eta_inc_s + eta_exit_s)
62         rp = (eta_exit_p - eta_inc_p) / (eta_exit_p + eta_inc_p)
63         return rs, rp
64
65
66 def theory_osc(nu, d_um, n1, theta0_deg, n2, n0=1.0):
67     # 双光束振荡项:  $R_{osc} = 2 r_{01}(1-r_{01}^2)r_{12} \cos(\delta)$ ,  $\delta = 4\pi n_1$ 
 $d \cos\theta_1 \cdot v/1e4$ 
68     nu = np.asarray(nu, float)
69     theta0 = np.deg2rad(theta0_deg)
70     theta1 = np.arcsin(np.minimum(n0 * np.sin(theta0) / n1,
1.0))
71     delta = 4.0 * np.pi * n1 * d_um * np.cos(theta1) * nu / 1
e4
72     e = np.exp(1j * delta)
73     rs01, rp01 = fresnel_r(n0, n1, theta0)
74     rs12, rp12 = fresnel_r(n1, n2, theta1)
75     Rs = np.abs(rs01 + (1 - rs01 ** 2) * rs12 * e) ** 2 - (
rs01 ** 2 + (1 - rs01 ** 2) ** 2 * rs12 ** 2)
76     Rp = np.abs(rp01 + (1 - rp01 ** 2) * rp12 * e) ** 2 - (
rp01 ** 2 + (1 - rp01 ** 2) ** 2 * rp12 ** 2)
77     return (Rs + Rp) / 2.0 * 100.0
78
79
80 def two_beam_reflectance(nu, d_um, n1, n2, theta0_deg, n0=1.0)
:
```

```

81     # 双光束完整反射率（含 DC），用于与 Airy 同口径对比
82     nu = np.asarray(nu, float)
83     theta0 = np.deg2rad(theta0_deg)
84     theta1 = np.arcsin(np.minimum(n0 * np.sin(theta0) / n1,
1.0))
85     delta = 4.0 * np.pi * n1 * d_um * np.cos(theta1) * nu / 1
e4
86     e = np.exp(1j * delta)
87     rs01, rp01 = fresnel_r(n0, n1, theta0)
88     rs12, rp12 = fresnel_r(n1, n2, theta1)
89     Rs = np.abs(rs01 + (1 - rs01 ** 2) * rs12 * e) ** 2
90     Rp = np.abs(rp01 + (1 - rp01 ** 2) * rp12 * e) ** 2
91     return (Rs + Rp) / 2.0 * 100.0
92
93
94 def theory_R(nu, d_um, beta, theta0_deg, n2, model, n0=1.0):
95     beta = np.asarray(beta, float)
96     if model == "cauchy":
97         n1 = n_cauchy(nu, *beta)
98     elif model == "airy":
99         n1 = n_cauchy(nu, *beta)
100     return airy_reflectance_osc(nu, d_um, n1, n2,
theta0_deg, n0)
101     elif model == "sellmeier":
102         n1 = n_sellmeier(nu, *beta)
103     elif model == "sellmeier1":
104         n1 = n_sellmeier1(nu, *beta)
105     elif model == "airy_sellmeier1":
106         n1 = n_sellmeier1(nu, *beta)
107     return airy_reflectance_osc(nu, d_um, n1, n2,
theta0_deg, n0)
108     else:
109         raise ValueError(f"未知模型: {model}")
110     return theory_osc(nu, d_um, n1, theta0_deg, n2, n0)
111

```

```

112
113 def airy_reflectance(nu, d_um, n1, n2, theta0_deg, n0=1.0):
114     # 多光束 Airy 完整反射率:  $r = (r_{01} + r_{12} e^{2i\delta}) / (1 + r_{01} r_{12} e^{2i\delta})$ 
115     nu = np.asarray(nu, float)
116     n1 = np.asarray(n1, complex)
117     n2 = np.asarray(n2, complex)
118     theta0 = np.deg2rad(theta0_deg)
119     cos0 = np.cos(theta0)
120     sin0 = n0 * np.sin(theta0)
121     sin1 = sin0 / n1
122     sin2 = sin0 / n2
123     cos1 = np.sqrt(1.0 - sin1 ** 2)
124     cos2 = np.sqrt(1.0 - sin2 ** 2)
125     delta = 2.0 * np.pi * nu * n1 * (d_um * 1e-4) * cos1
126     e2 = np.exp(2j * delta)
127
128     def _r_pol(eta0, eta1, eta2):
129         r01 = (eta0 - eta1) / (eta0 + eta1)
130         r12 = (eta1 - eta2) / (eta1 + eta2)
131         return (r01 + r12 * e2) / (1.0 + r01 * r12 * e2)
132
133     rs = _r_pol(n0 * cos0, n1 * cos1, n2 * cos2)
134     rp = _r_pol(n0 / cos0, n1 / cos1, n2 / cos2)
135     R = (np.abs(rs) ** 2 + np.abs(rp) ** 2) / 2.0
136     return R * 100.0
137
138
139 def airy_reflectance_avg(nu, d_um, sigma_d, n1, n2, theta0_deg
140     , n0=1.0, n_gh=9):
141     # 光斑内厚度高斯分布系综平均 (Gauss-Hermite)
142     nu = np.asarray(nu, float)
143     if sigma_d <= 0.0:
144         return airy_reflectance(nu, d_um, n1, n2, theta0_deg,
145             n0)

```



```

144     x, w = np.polynomial.hermite.hermgauss(n_gh)
145     R = np.zeros_like(nu)
146     for xi, wi in zip(x, w):
147         d_i = d_um + np.sqrt(2.0) * sigma_d * xi
148         R = R + wi * airy_reflectance(nu, d_i, n1, n2,
theta0_deg, n0)
149     return R / np.sqrt(np.pi)
150
151
152 def airy_reflectance_osc(nu, d_um, n1, n2, theta0_deg, n0=1.0)
:
153     # Airy 振荡项 = 完整反射率 - 解析 DC 项 (与 theory_osc 同
口径)
154     R = airy_reflectance(nu, d_um, n1, n2, theta0_deg, n0)
155     theta0 = np.deg2rad(theta0_deg)
156     theta1 = np.arcsin(np.minimum(n0 * np.sin(theta0) / np.
asarray(n1, float), 1.0))
157     rs01, rp01 = fresnel_r(n0, n1, theta0)
158     rs12, rp12 = fresnel_r(n1, n2, theta1)
159     dc = ((rs01 ** 2 + (1 - rs01 ** 2) ** 2 * rs12 ** 2)
160           + (rp01 ** 2 + (1 - rp01 ** 2) ** 2 * rp12 ** 2)) /
2.0
161     return R - dc * 100.0
162
163
164 def two_beam_reflectance_avg(nu, d_um, sigma_d, n1, n2,
theta0_deg, n0=1.0, n_gh=9):
165     nu = np.asarray(nu, float)
166     if sigma_d <= 0.0:
167         return two_beam_reflectance(nu, d_um, n1, n2,
theta0_deg, n0)
168     x, w = np.polynomial.hermite.hermgauss(n_gh)
169     R = np.zeros_like(nu)
170     for xi, wi in zip(x, w):
171         d_i = d_um + np.sqrt(2.0) * sigma_d * xi

```

```

172         R = R + wi * two_beam_reflectance(nu, d_i, n1, n2,
theta0_deg, n0)
173     return R / np.sqrt(np.pi)

```

preprocess.py

```

1  from __future__ import annotations
2
3  import numpy as np
4  import pandas as pd
5  from scipy import sparse
6  from scipy.ndimage import median_filter
7  from scipy.sparse.linalg import spsolve
8
9
10 def load_spectra(path):
11     # 附件格式：第1列波数( $\text{cm}^{-1}$ )、第2列反射率(%)
12     df = pd.read_excel(path, header=0)
13     wn = df.iloc[:, 0].to_numpy(dtype=float)
14     R = df.iloc[:, 1].to_numpy(dtype=float)
15     return wn, R
16
17
18 def find_cutoff_stft(wn, R, window=1200, hop=200, eta=0.3,
19                     f0_est=(1500.0, 4000.0), dnu_band=4,
20                     detrend_win=1000):
21     # 滑动窗口 FFT：主频带能量/低频残留能量 达峰值 eta 倍处为
22     # 裁剪分界点
23     wn = np.asarray(wn, float)
24     R = np.asarray(R, float)
25     dnu = float(np.median(np.diff(wn)))
26
27     bg = np.convolve(R, np.ones(detrend_win) / detrend_win,
mode="same")
28     Rd = R - bg

```

```

28     sel = (wn >= f0_est[0]) & (wn <= f0_est[1])
29     if np.sum(sel) < 8:
30         return float(wn[0])
31     sig = Rd[sel] * np.hanning(np.sum(sel))
32     F = np.fft.rfft(sig)
33     freqs = np.fft.rfftfreq(len(sig), d=dnu)
34     f0 = float(freqs[1 + int(np.argmax(np.abs(F[1:])))])
35
36     n = len(wn)
37     fw = np.fft.rfftfreq(window, d=dnu)
38     f0_idx = int(np.argmin(np.abs(fw - f0)))
39     band = slice(max(1, f0_idx - dnu_band), min(len(fw),
f0_idx + dnu_band + 1))
40     low = slice(1, max(2, f0_idx // 2))
41
42     centers, ratios = [], []
43     for start in range(0, n - window + 1, hop):
44         wsig = Rd[start:start + window]
45         wsig = (wsig - np.mean(wsig)) * np.hanning(window)
46         Fw = np.abs(np.fft.rfft(wsig)) ** 2
47         Ef = float(np.sum(Fw[band]))
48         El = float(np.sum(Fw[low]))
49         ratios.append(Ef / max(El, 1e-12))
50         centers.append(float(wn[start + window // 2]))
51
52     ratios = np.asarray(ratios)
53     peak = ratios.max()
54     if peak <= 0:
55         return float(wn[0])
56     ok = np.where(ratios >= eta * peak)[0]
57     return float(centers[ok[0]]) if len(ok) else float(wn[0])
58
59
60 def resample(wn, R, step=None):
61     wn = np.asarray(wn, float)

```

```

62     R = np.asarray(R, float)
63     if step is None:
64         step = float(np.median(np.diff(wn)))
65     wn_new = np.arange(wn.min(), wn.max() + step, step)
66     R_new = np.interp(wn_new, wn, R)
67     return wn_new, R_new
68
69
70 def hampel_filter(R, win=11, t=3.0):
71     # 偏离滑动中位数超过  $t \cdot \sigma_{MAD}$  的点用中位数替换
72     R = np.asarray(R, float)
73     med = median_filter(R, size=win, mode="reflect")
74     resid = R - med
75     mad = median_filter(np.abs(resid), size=win, mode="reflect")
76     sigma = 1.4826 * mad
77     sigma[sigma < 1e-12] = 1e-12
78     mask = np.abs(resid) > t * sigma
79     out = R.copy()
80     out[mask] = med[mask]
81     return out
82
83
84 def asls_baseline(R, lam=1e5, p=0.01, niter=10):
85     R = np.asarray(R, float)
86     L = len(R)
87     D = sparse.diags([1.0, -2.0, 1.0], [0, 1, 2], shape=(L -
88 2, L))
89     DtD = (D.T @ D).tocsc()
90     w = np.ones(L)
91     for _ in range(niter):
92         W = sparse.diags(w, 0, format="csc")
93         Z = W + lam * DtD
94         z = spsolve(Z, w * R)
95         w = np.where(R > z, p, 1.0 - p)

```

```

95     return z
96
97
98 def sg_smooth(R, win=15, polyorder=3):
99     from scipy.signal import savgol_filter
100     return savgol_filter(np.asarray(R, float), window_length=
        win, polyorder=polyorder)
101
102
103 def preprocess(wn, R, cutoff=None, step=None,
104               hampel_win=11, hampel_t=3.0,
105               asls_lam=1e5, asls_p=0.01,
106               sg_win=15, sg_poly=3):
107     wn = np.asarray(wn, float)
108     R = np.asarray(R, float)
109
110     if cutoff is None:
111         cutoff = find_cutoff_stft(wn, R)
112
113     mask = wn >= cutoff
114     wn, R = wn[mask], R[mask]
115
116     wn, R = resample(wn, R, step)
117
118     R = hampel_filter(R, win=hampel_win, t=hampel_t)
119     baseline = asls_baseline(R, lam=asls_lam, p=asls_p)
120     R = R - baseline
121     R = sg_smooth(R, win=sg_win, polyorder=sg_poly)
122     dc = float(np.mean(R))
123     R = R - dc
124
125     return wn, R, float(cutoff), baseline, dc

```

inversion.py

```

1 from __future__ import annotations

```

```

2
3 from dataclasses import dataclass, field
4
5 import numpy as np
6 from scipy import linalg
7 from scipy.optimize import differential_evolution,
    least_squares
8 from scipy.stats import t as t_dist
9
10 from optics import theory_R
11
12
13 @dataclass
14 class FitResult:
15     model: str
16     theta: np.ndarray
17     d: float
18     beta: np.ndarray
19     linear: dict
20     n_lin: int = 0
21     res: object = None
22     residual1: np.ndarray = field(default_factory=lambda: np.
array([]))
23     residual2: np.ndarray = field(default_factory=lambda: np.
array([]))
24     fit1: np.ndarray = field(default_factory=lambda: np.array
([]))
25     fit2: np.ndarray = field(default_factory=lambda: np.array
([]))
26
27     @property
28     def n_params(self) -> int:
29         return len(self.theta)
30
31

```

```

32 def fft_init_d(wn, R, n_avg=2.6, theta0_deg=10.0, search_lo
    =1000.0):
33     #  $d_0 = f_0 \cdot 1e4 / (2 \cdot \pi \cdot \cos \theta_0)$ 
34     wn = np.asarray(wn, float)
35     R = np.asarray(R, float)
36     sel = wn >= search_lo
37     wn, R = wn[sel], R[sel]
38     dnu = float(np.median(np.diff(wn)))
39     n_pts = len(R)
40
41     sig = (R - R.mean()) * np.hanning(n_pts)
42     F = np.fft.rfft(sig)
43     freqs = np.fft.rfftfreq(n_pts, d=dnu)
44     idx = 1 + int(np.argmax(np.abs(F[1:])))
45
46     if 1 <= idx < len(freqs) - 1:
47         y0, y1, y2 = np.abs(F[idx - 1]), np.abs(F[idx]), np.
abs(F[idx + 1])
48         denom = y0 - 2.0 * y1 + y2
49         if abs(denom) > 1e-12:
50             idx = idx + 0.5 * (y0 - y2) / denom
51         f0 = float(freqs[int(idx)]) if idx == int(idx) else float(
52             np.interp(idx, np.arange(len(freqs)), freqs))
53
54     theta1 = np.arcsin(np.sin(np.deg2rad(theta0_deg)) / n_avg)
55     return float(f0 * 1e4 / (2.0 * n_avg * np.cos(theta1)))
56
57
58 def _lin_solve(T, R):
59     A = np.column_stack([T, np.ones_like(T)])
60     sol, *_ = np.linalg.lstsq(A, R, rcond=None)
61     return float(sol[0]), float(sol[1])
62
63
64 def de_init_beta(wn1, R1, wn2, R2, d0, model, n2, angles,

```

```

        bounds_beta, seed=0):
65     def objective(beta):
66         T1 = theory_R(wn1, d0, beta, angles[0], n2, model)
67         a1, b1 = _lin_solve(T1, R1)
68         e1 = a1 * T1 + b1 - R1
69         T2 = theory_R(wn2, d0, beta, angles[1], n2, model)
70         a2, b2 = _lin_solve(T2, R2)
71         e2 = a2 * T2 + b2 - R2
72         return float(np.sum(e1 ** 2) + np.sum(e2 ** 2))
73
74     res = differential_evolution(
75         objective, bounds_beta, seed=seed, tol=1e-8,
76         popsize=20, maxiter=200, polish=True, updating="
immediate",
77     )
78     return np.asarray(res.x, float)
79
80
81 def lm_fit(wn1, R1, wn2, R2, d0, beta0, model, n2, angles,
            bounds, seed=0, fix_d=False):
82     use2 = R2 is not None
83     if fix_d:
84         theta0 = np.asarray(beta0, float)
85     else:
86         theta0 = np.r_[d0, np.asarray(beta0, float)]
87
88     def residuals(th):
89         if fix_d:
90             d, beta = d0, th
91         else:
92             d, beta = th[0], th[1:]
93         T1 = theory_R(wn1, d, beta, angles[0], n2, model)
94         a1, b1 = _lin_solve(T1, R1)
95         e1 = a1 * T1 + b1 - R1
96         if not use2:

```



```

97         return e1
98     T2 = theory_R(wn2, d, beta, angles[1], n2, model)
99     a2, b2 = _lin_solve(T2, R2)
100     e2 = a2 * T2 + b2 - R2
101     return np.concatenate([e1, e2])
102
103     bd = bounds[1:] if fix_d else bounds
104     lb = np.array([b[0] for b in bd], float)
105     ub = np.array([b[1] for b in bd], float)
106     res = least_squares(residuals, theta0, bounds=(lb, ub),
method="trf",
107                        xtol=1e-12, ftol=1e-12, gtol=1e-12,
max_nfev=1000)
108
109     if fix_d:
110         theta = np.r_[d0, res.x]
111         d, beta = d0, res.x
112     else:
113         theta = res.x
114         d, beta = theta[0], theta[1:]
115     T1 = theory_R(wn1, d, beta, angles[0], n2, model)
116     a1, b1 = _lin_solve(T1, R1)
117     fit1 = a1 * T1 + b1
118     e1 = fit1 - R1
119     linear = {angles[0]: (a1, b1)}
120     if use2:
121         T2 = theory_R(wn2, d, beta, angles[1], n2, model)
122         a2, b2 = _lin_solve(T2, R2)
123         fit2 = a2 * T2 + b2
124         e2 = fit2 - R2
125         linear[angles[1]] = (a2, b2)
126     else:
127         fit2 = np.array([])
128         e2 = np.array([])
129     return FitResult(model=model, theta=theta, d=float(d),

```

```

beta=beta,
130         linear=linear, n_lin=2 * (2 if use2 else
1), res=res,
131         residual1=e1, residual2=e2, fit1=fit1,
fit2=fit2)
132
133
134 def fit_statistics(res, n_params, R1, R2=None):
135     fun = res.fun
136     n_data = len(fun)
137     SSE = float(np.sum(fun ** 2))
138     rmse = float(np.sqrt(SSE / n_data))
139     obs = np.concatenate([R1, R2]) if R2 is not None else R1
140     ss_tot = float(np.sum((obs - obs.mean()) ** 2))
141     r2 = float(1.0 - SSE / ss_tot) if ss_tot > 0 else float("
nan")
142     aic = float(n_data * np.log(SSE / n_data) + 2 * n_params)
143     bic = float(n_data * np.log(SSE / n_data) + n_params * np.
log(n_data))
144     return {"n_data": n_data, "SSE": SSE, "RMSE": rmse, "R2":
r2, "AIC": aic, "BIC": bic}
145
146
147 def parameter_uncertainty(res, param_names, n_lin=0):
148     J = res.jac
149     m, n = J.shape
150     dof = m - n - n_lin
151     s2 = float(np.sum(res.fun ** 2) / max(dof, 1))
152     cov = s2 * linalg.pinv(J.T @ J)
153     sd = np.sqrt(np.maximum(np.diag(cov), 0.0))
154     tval = float(t_dist.ppf(0.975, max(dof, 1)))
155     ci = np.column_stack([res.x - tval * sd, res.x + tval * sd
])
156     return sd, ci, {"sigma2": s2, "t_value": tval, "dof": dof,
"cov": cov}

```

```

157
158
159 def noise_robustness_test(wn1, R1, wn2, R2, model, n2, angles,
160                           bounds, de_bounds,
161                           levels=(0.01, 0.02, 0.05), nrep=5,
162                           seed=0, search_lo=1500.0,
163                           profile_step=0.05, d_half=0.5):
164     rng = np.random.default_rng(seed)
165     out = {}
166     for eta in levels:
167         ds = []
168         for rep in range(nrep):
169             eps1 = rng.normal(0.0, eta * R1.std(), R1.shape)
170             eps2 = rng.normal(0.0, eta * R2.std(), R2.shape)
171             R1n, R2n = R1 + eps1, R2 + eps2
172             d0n = fft_init_d(wn1, R1n, n_avg=2.6, theta0_deg=
angles[0], search_lo=search_lo)
173             beta0n = de_init_beta(wn1, R1n, wn2, R2n, d0n,
model, n2, angles,
174                                   de_bounds, seed=seed + rep)
175             fit, _ = profile_scan_d(wn1, R1n, wn2, R2n, d0n,
beta0n, model, n2,
176                                   angles, bounds, d_half=
d_half,
177                                   step=profile_step, seed=
seed + rep)
178             ds.append(fit.d)
179             out[str(eta)] = {"d_mean": float(np.mean(ds)), "d_std"
: float(np.std(ds)),
180                             "d_min": float(np.min(ds)), "d_max":
float(np.max(ds))}
181         return out
182
183
184 def single_angle_cross_check(wn1, R1, wn2, R2, d0, beta0,

```

```

model, n2, angles, bounds, seed=0):
185     fit1 = lm_fit(wn1, R1, None, None, d0, beta0, model, n2,
angles, bounds, seed=seed)
186     fit2 = lm_fit(wn2, R2, None, None, d0, beta0, model, n2,
angles, bounds, seed=seed)
187     return {"d_angle1": float(fit1.d), "d_angle2": float(fit2.
d)}
188
189
190 def profile_scan_d(wn1, R1, wn2, R2, d0, beta0, model, n2,
angles, bounds,
191                     d_half=0.5, step=0.01, seed=0):
192     d_grid = np.arange(d0 - d_half, d0 + d_half + step * 0.5,
step)
193     sse = np.full_like(d_grid, np.inf)
194     best_fit = None
195     best_sse = np.inf
196     cur_beta = beta0
197     for i, d in enumerate(d_grid):
198         fit = lm_fit(wn1, R1, wn2, R2, float(d), cur_beta,
model, n2, angles,
199                     bounds, seed=seed, fix_d=True)
200         if any(v[0] <= 0.0 for v in fit.linear.values()): #
正增益约束, 排除反相分支
201             continue
202         sse[i] = float(np.sum(fit.res.fun ** 2))
203         if sse[i] < best_sse:
204             best_sse = sse[i]
205             best_fit = fit
206             cur_beta = fit.beta
207
208     if best_fit is None:
209         best_fit = lm_fit(wn1, R1, wn2, R2, float(d0), beta0,
model, n2,
210                         angles, bounds, seed=seed, fix_d=

```

```

True)
211     sse[:] = float(np.sum(best_fit.res.fun ** 2))
212     best_sse = sse[0]
213     d_opt = float(d0)
214     else:
215         valid = np.isfinite(sse)
216         k = int(np.argmin(np.where(valid, sse, np.inf)))
217         if 0 < k < len(d_grid) - 1 and np.isfinite(sse[k - 1])
and np.isfinite(sse[k + 1]):
218             y0, y1, y2 = sse[k - 1], sse[k], sse[k + 1]
219             denom = y0 - 2.0 * y1 + y2
220             if abs(denom) > 1e-15:
221                 d_opt = d_grid[k] + 0.5 * step * (y0 - y2) /
denom
222             else:
223                 d_opt = d_grid[k]
224             else:
225                 d_opt = d_grid[k]
226                 final_fit = lm_fit(wn1, R1, wn2, R2, float(d_opt),
cur_beta, model, n2,
227                                     angles, bounds, seed=seed, fix_d=
True)
228                 if all(v[0] > 0.0 for v in final_fit.linear.values()):
229                     best_fit = final_fit
230
231     sse_min = float(np.sum(best_fit.res.fun ** 2))
232
233     m = len(best_fit.res.fun)
234     n_beta = best_fit.res.x.size
235     n_lin = best_fit.n_lin
236     dof = m - n_beta - n_lin - 1
237     s2 = sse_min / max(dof, 1)
238     valid = np.isfinite(sse)
239     valid_idx = np.where(valid)[0]
240     k = int(np.argmin(np.where(valid, sse, np.inf)))

```

```

241     lo_k = max(valid_idx.min(), k - 3)
242     hi_k = min(valid_idx.max() + 1, k + 4)
243     dloc = d_grid[lo_k:hi_k]
244     sloc = sse[lo_k:hi_k]
245     dloc = dloc[np.isfinite(sloc)]
246     sloc = sloc[np.isfinite(sloc)]
247     a2 = 0.0
248     if len(dloc) >= 3:
249         c = np.polyfit(dloc, sloc, 2)
250         a2 = float(c[0])
251     if a2 > 1e-12:
252         std_d = float(np.sqrt(s2 / a2))
253     else:
254         std_d = float(step)
255     tval = float(t_dist.ppf(0.975, max(dof, 1)))
256     d_ci = (float(d_opt - tval * std_d), float(d_opt + tval *
std_d))
257
258     profile = {
259         "d_grid": d_grid.tolist(), "sse": sse.tolist(),
260         "sse_min": sse_min, "d_opt": float(d_opt),
261         "std_d": std_d, "d_ci": list(d_ci),
262     }
263     return best_fit, profile

```

q2.py

```

1  from __future__ import annotations
2
3  import json
4  import sys
5  from pathlib import Path
6
7  import numpy as np
8  import yaml
9

```

```

10 from preprocess import load_spectra, preprocess
11 from inversion import (de_init_beta, fft_init_d,
12     fit_statistics,
13     noise_robustness_test,
14     parameter_uncertainty,
15     profile_scan_d,
16     single_angle_cross_check)
17
18 N0 = 1.0
19 N2 = 2.65
20 ANG1, ANG2 = 10.0, 15.0
21 ANGLES = (ANG1, ANG2)
22
23 PREPROC = dict(hampel_win=11, hampel_t=3.0, asls_lam=1e5,
24     asls_p=0.01,
25     sg_win=15, sg_poly=3)
26
27 BOUNDS = {
28     "cauchy": [(0.5, 200.0), (1.5, 3.5), (-1.0, 1.0), (-1.0,
29     1.0)],
30     "sellmeier": [(0.5, 200.0), (0.0, 3.0), (0.1, 2000.0),
31     (0.0, 3.0), (0.1, 2000.0)],
32     "sellmeier1": [(0.5, 200.0), (0.0, 8.0), (0.0, 1.0)],
33 }
34
35 DE_BOUNDS = {
36     "cauchy": [(1.5, 3.5), (-1.0, 1.0), (-1.0, 1.0)],
37     "sellmeier": [(0.0, 3.0), (0.1, 2000.0), (0.0, 3.0), (0.1,
38     2000.0)],
39     "sellmeier1": [(0.0, 8.0), (0.0, 1.0)],
40 }
41
42 PARAM_NAMES = {
43     "cauchy": ["d( $\mu\text{m}$ )", "A", "B", "C"],
44     "sellmeier": ["d( $\mu\text{m}$ )", "B1", "C1", "B2", "C2"],
45     "sellmeier1": ["d( $\mu\text{m}$ )", "B", "C"],
46 }

```

```

38 SEED = 2025
39
40
41 def load_config():
42     root = Path(__file__).resolve().parents[2]
43     with open(root / "config.yaml", encoding="utf-8") as f:
44         cfg = yaml.safe_load(f)
45     return root, cfg
46
47
48 def save_csv(path, header, cols):
49     arr = np.column_stack(cols)
50     fmt = ", ".join(["%.6f"] * arr.shape[1])
51     np.savetxt(path, arr, fmt=fmt, header=header, comments="")
52     print(f" [输出] {path.name} ({arr.shape[0]} 行)")
53
54
55 def save_json(path, data):
56     with open(path, "w", encoding="utf-8") as f:
57         json.dump(data, f, ensure_ascii=False, indent=2)
58     print(f" [输出] {path.name}")
59
60
61 def solve_model(wn1, R1, wn2, R2, d0, model, d_half=0.5, step
    =0.01):
62     # DE 固定 d0 搜色散初值 → 剖面扫描精调 d (锁在 FFT 分支
    内)
63     beta0 = de_init_beta(wn1, R1, wn2, R2, d0, model, N2,
    ANGLES,
64                             DE_BOUNDS[model], seed=SEED)
65     fit, profile = profile_scan_d(wn1, R1, wn2, R2, d0, beta0,
    model, N2,
66                                     ANGLES, BOUNDS[model],
    d_half=d_half,
67                                     step=step, seed=SEED)

```



```

68     d = fit.d
69     stats = fit_statistics(fit.res, fit.res.x.size + fit.n_lin
, R1, R2)
70     pnames = PARAM_NAMES[model][1:]
71     sd, ci, _ = parameter_uncertainty(fit.res, pnames, n_lin=
fit.n_lin)
72     cross = single_angle_cross_check(wn1, R1, wn2, R2, d,
beta0, model,
73                                     N2, ANGLES, BOUNDS[model
], seed=SEED)
74     linear = {f"angle{int(k)}": {"a": v[0], "b": v[1]}
75               for k, v in fit.linear.items()}
76     summary = {
77         "model": model,
78         "theta": fit.theta.tolist(),
79         "d": d, "d0": d0,
80         "d_ci": profile["d_ci"],
81         "d_std": profile["std_d"],
82         "profile_sse_min": profile["sse_min"],
83         "profile_d_grid": profile["d_grid"],
84         "profile_sse": profile["sse"],
85         "beta": fit.beta.tolist(),
86         "linear": linear,
87         "param_sd": sd.tolist(),
88         "param_ci": ci.tolist(),
89         "param_names": pnames,
90         "stats": stats,
91         "cross_check": cross,
92         "beta0": beta0.tolist(),
93     }
94     return summary, fit
95
96
97 def main():
98     root, cfg = load_config()

```

```

99     raw = cfg["data"]["raw_files"]
100     result_dir = root / cfg["result"]["dir"]
101     result_dir.mkdir(parents=True, exist_ok=True)
102
103     print("=" * 60)
104     print("问题二：碳化硅外延层厚度反演（双光束干涉模型）")
105     print("=" * 60)
106
107     print("\n[1] 数据读取与预处理")
108     spec = {}
109     for key, path in [("f1", raw["f1"]), ("f2", raw["f2"])] :
110         wn, R = load_spectra(root / path)
111         wn_p, R_p, cutoff, baseline, dc = preprocess(wn, R, **
122 PREPROC)
112         if not (700.0 <= cutoff <= 1600.0):
113             cutoff = 1000.0
114             wn_p, R_p, _, baseline, dc = preprocess(wn, R,
125 cutoff=cutoff, **PREPROC)
115             spec[key] = (wn_p, R_p)
116             print(f" {key}: {len(wn)} 点 -> 裁剪点 {cutoff:.1f}
127 cm-1 -> {len(wn_p)} 点")
117             save_csv(result_dir / f"q2_processed_{key}.csv",
118                     "wavenumber_cm-1,reflectance_osc_%,baseline_
129 %,dc_%",
119                     [wn_p, R_p, baseline, np.full(len(baseline),
130 dc)])
131
132     wn1, R1 = spec["f1"]
133     wn2, R2 = spec["f2"]
134
135     print("\n[2] FFT 厚度初值估计")
136     d0 = fft_init_d(wn1, R1, n_avg=2.6, theta0_deg=ANG1,
137 search_lo=1500.0)
138     print(f" d0 = {d0:.3f} um")

```

```

128     print("\n[3] 模型求解 (DE 初值 + 剖面扫描精调 d)")
129     results = {}
130     for model in ["cauchy", "sellmeier1"]:
131         print(f"    --- 模型: {model} ---")
132         results[model], fit = solve_model(wn1, R1, wn2, R2, d0
, model)
133         r = results[model]
134         lin = r["linear"]
135         d_ci = r["d_ci"]
136         print(f"        d = {r['d']:.4f} um    95%CI [{d_ci[0]:.4f},
{d_ci[1]:.4f}]    "
137             f"RMSE = {r['stats']['RMSE']:.4f}    R^2 = {r['
stats']['R2']:.4f}")
138         print(f"        线性校正 10deg: a={lin['angle10']['a']:.4f
} b={lin['angle10']['b']:.4f}    |    "
139             f"15deg: a={lin['angle15']['a']:.4f} b={lin['
angle15']['b']:.4f}")
140         print(f"        单角度独立拟合(自由d): 10deg -> {r['
cross_check']['d_angle1']:.4f} um, "
141             f"15deg -> {r['cross_check']['d_angle2']:.4f} um
")
142         save_csv(result_dir / f"q2_fit_{model}.csv",
143                 "wn1,R_obs1,R_fit1,wn2,R_obs2,R_fit2",
144                 [wn1, R1, fit.fit1, wn2, R2, fit.fit2])
145         names = PARAM_NAMES[model]
146         theta_full = np.concatenate([[r["d"]], np.asarray(r["
beta"], float))])
147         sd_full = np.concatenate([[r["d_std"]], np.asarray(r["
param_sd"], float))])
148         ci_lo = np.concatenate([[d_ci[0]], np.asarray(r["
param_ci"], float)[: , 0])])
149         ci_hi = np.concatenate([[d_ci[1]], np.asarray(r["
param_ci"], float)[: , 1])])
150         param_csv = np.column_stack([np.array(names, dtype=
object),

```

```

151                                     np.round(theta_full, 6),
np.round(sd_full, 6),
152                                     np.round(ci_lo, 6), np.
round(ci_hi, 6)])
153     np.savetxt(result_dir / f"q2_parameters_{model}.csv",
param_csv,
154                 fmt="%s", delimiter=",",
155                 header="param,value,sd,ci_lo,ci_hi",
comments="")
156     print(f"  [输出] q2_parameters_{model}.csv")
157
158     print("\n[4] 抗噪声能力测试（全局反演，含 DE，较慢）")
159     noise = {}
160     for model in ["cauchy", "sellmeier1"]:
161         noise[model] = noise_robustness_test(
162             wn1, R1, wn2, R2, model, N2, ANGLES,
163             BOUNDS[model], DE_BOUNDS[model],
164             levels=(0.01, 0.02, 0.03, 0.04, 0.05), nrep=5,
seed=SEED,
165             profile_step=0.05, d_half=2.0)
166     for eta, v in noise[model].items():
167         print(f"  {model} eta={eta}: d = {v['d_mean']:.4f}
+/- {v['d_std']:.4f} um")
168     save_json(result_dir / "q2_noise_test.json", noise)
169
170     summary = {"d0_fft": d0, "N0": N0, "N2": N2,
171               "angles": {"f1": ANG1, "f2": ANG2},
172               "models": results}
173     save_json(result_dir / "q2_results.json", summary)
174
175     print("\n" + "=" * 60)
176     print("两模型结果对比")
177     print("=" * 60)
178     print(f"{'指标':<10}{'Cauchy':>14}{'Sellmeier1':>14}")
179     for key, label in [("d", "d (um)"), ("RMSE", "RMSE"), ("R2

```

```

180         ("AIC", "AIC"), ("BIC", "BIC")]:
181     if key in ("RMSE", "R2", "AIC", "BIC"):
182         row = f"{label:<10}"
183         for m in ["cauchy", "sellmeier1"]:
184             row += f"{results[m]['stats'][key]:>14.4f}"
185     else:
186         row = f"{label:<10}"
187         for m in ["cauchy", "sellmeier1"]:
188             row += f"{results[m][key]:>14.4f}"
189     print(row)
190     print("\n完成。结果已写入 outputs/result/ 目录。")
191
192
193 if __name__ == "__main__":
194     sys.exit(main())

```

q3.py

```

1 from __future__ import annotations
2 import json
3 from pathlib import Path
4 import numpy as np
5 import yaml
6 from scipy.optimize import differential_evolution,
    least_squares
7 from preprocess import (find_cutoff_stft, hampel_filter,
    load_spectra,
8                             resample, sg_smooth, preprocess)
9 from optics import (airy_reflectance_avg, n_drude,
    n_sellmeier1,
10                        two_beam_reflectance_avg)
11 from inversion import (de_init_beta, fft_init_d,
    fit_statistics,
12                        parameter_uncertainty, profile_scan_d)
13

```

```

14 N0 = 1.0
15 ANG1, ANG2 = 10.0, 15.0
16 ANGLES = (ANG1, ANG2)
17
18 N_SI_AVG = 3.42
19 SI_CUTOFF = 2000.0
20 PREPROC = dict(hampel_win=11, hampel_t=3.0, sg_win=15, sg_poly
    =3)
21
22 # 硅晶圆片外延层 n1 固定（无色散硅）： $n_1^2=1+B$ ,  $B=10.7$ ,  $C=0 \rightarrow$ 
     $n_1 \approx 3.4205$ 
23 # 衬底 n2 用 Drude 色散； $\sigma_d=0$  取理想平行平面（多光束判定基
    准）
24 SI_B = 10.7
25 SI_C = 0.0
26 SI_N1 = float(np.sqrt(1.0 + SI_B))
27
28 TWO_NAMES = ["d(um)", "n_inf", "wp", "gamma"]
29 TWO_BOUNDS = [(0.5, 200.0), (2.5, 3.4), (300.0, 8000.0), (5.0,
    500.0)]
30 TWO_DE_BOUNDS = [(2.5, 3.4), (300.0, 8000.0), (5.0, 500.0)]
31
32 AIRY_NAMES = ["d(um)", "n_inf", "wp", "gamma"]
33 AIRY_BOUNDS = [(0.5, 200.0), (2.5, 3.4), (300.0, 8000.0),
    (5.0, 500.0)]
34 AIRY_DE_BOUNDS = [(2.5, 3.4), (300.0, 8000.0), (5.0, 500.0)]
35
36 SIC_CUTOFF_FALLBACK = 1000.0
37 N2_SIC = 2.65
38 SIC_PREPROC = dict(hampel_win=11, hampel_t=3.0, asls_lam=1e5,
    asls_p=0.01,
39                     sg_win=15, sg_poly=3)
40
41 SIC_NAMES_TWO = ["d(um)", "B", "C"]
42 SIC_NAMES_AIRY = ["d(um)", "B", "C"]

```

```

43 SIC_SELLMEIER1_BOUNDS = [(0.0, 8.0), (0.0, 1.0)]
44
45 SEED = 2025
46
47 def load_config():
48     root = Path(__file__).resolve().parents[2]
49     with open(root / "config.yaml", encoding="utf-8") as f:
50         cfg = yaml.safe_load(f)
51     return root, cfg
52
53 def save_json(path, data):
54     with open(path, "w", encoding="utf-8") as f:
55         json.dump(data, f, ensure_ascii=False, indent=2)
56     print(f" [output] {path.name}")
57
58 def save_csv(path, header, cols):
59     arr = np.column_stack(cols)
60     fmt = ",".join(["%.6f"] * arr.shape[1])
61     np.savetxt(path, arr, fmt=fmt, header=header, comments="")
62     print(f" [output] {path.name} ({arr.shape[0]} rows)")
63
64 def preprocess_si(wn, R, cutoff):
65     # 硅晶圆片：裁剪 → 重采样 → Hampel → SG（保留 DC）
66     mask = wn >= cutoff
67     wn, R = wn[mask], R[mask]
68     wn, R = resample(wn, R)
69     R = hampel_filter(R, win=PREPROC["hampel_win"], t=PREPROC[
70         "hampel_t"])
71     R = sg_smooth(R, win=PREPROC["sg_win"], polyorder=PREPROC[
72         "sg_poly"])
73     return wn, R
74
75 def preprocess_sic(wn, R):
76     cutoff = find_cutoff_stft(wn, R)
77     if not (700.0 <= cutoff <= 1600.0):

```

```

76         cutoff = SIC_CUTOFF_FALLBACK
77     wn_p, R_p = preprocess_si(wn, R, cutoff)
78     return wn_p, R_p, cutoff
79
80 def airy_R(nu, theta_deg, theta):
81     d, n_inf, wp, gamma = theta
82     n1 = n_sellmeier1(nu, SI_B, SI_C)
83     n2 = n_drude(nu, n_inf, wp, gamma)
84     return airy_reflectance_avg(nu, d, 0.0, n1, n2, theta_deg,
85                                N0)
86
87 def twobeam_R(nu, theta_deg, theta):
88     d, n_inf, wp, gamma = theta
89     n1 = n_sellmeier1(nu, SI_B, SI_C)
90     n2 = n_drude(nu, n_inf, wp, gamma)
91     return two_beam_reflectance_avg(nu, d, 0.0, n1, n2,
92                                     theta_deg, N0)
93
94 def make_residual(wn1, R1, wn2, R2, model_func):
95     # 每角度独立 DC 偏置 b（闭式消元，无增益），避免增益与折射
96     # 率耦合的退化解
97     def resid(theta):
98         T1 = model_func(wn1, ANG1, theta)
99         b1 = float(np.mean(R1 - T1))
100         e1 = T1 + b1 - R1
101         T2 = model_func(wn2, ANG2, theta)
102         b2 = float(np.mean(R2 - T2))
103         e2 = T2 + b2 - R2
104         return np.concatenate([e1, e2])
105     return resid
106
107 def fit_model(wn1, R1, wn2, R2, d0, model_func, bounds,
108               de_bounds, seed=SEED):
109     resid = make_residual(wn1, R1, wn2, R2, model_func)

```



```

107     def sse_other(x):
108         return float(np.sum(resid(np.r_[d0, x]) ** 2))
109
110     de = differential_evolution(sse_other, de_bounds, seed=
seed,
111                                tol=1e-8, popsize=20, maxiter
=200,
112                                polish=True, updating="
immediate")
113     x0 = np.r_[d0, de.x]
114     lb = np.array([b[0] for b in bounds], float)
115     ub = np.array([b[1] for b in bounds], float)
116     res = least_squares(resid, x0, bounds=(lb, ub), method="
trf",
117                        xtol=1e-12, ftol=1e-12, gtol=1e-12,
max_nfev=3000)
118     return res, de.x
119
120 def residual_harmonics(wn, resid, f0, nharm=3, half_peak=3.0):
121     # H_k = k * f0 谐波峰幅值 / 峰两侧局部邻带噪声标准差
122     dnu = float(np.median(np.diff(wn)))
123     F = np.abs(np.fft.rfft(resid * np.hanning(len(resid))))
124     freqs = np.fft.rfftfreq(len(resid), d=dnu)
125     dfreq = float(freqs[1] - freqs[0]) if len(freqs) > 1 else
0.0
126     out = {}
127     for k in range(2, nharm + 1):
128         target = k * f0
129         half = half_peak * dfreq
130         sel_peak = np.abs(freqs - target) <= half
131         if not np.any(sel_peak):
132             out[k] = 0.0
133             continue
134         peak = float(F[sel_peak].max())
135         sel_noise = (np.abs(freqs - target) > half) & \

```

```

136         (np.abs(freqs - target) <= 3.0 * half)
137         sigma = float(F[sel_noise].std()) if np.sum(sel_noise)
138         > 2 else 1.0
139         out[k] = peak / max(sigma, 1e-12)
140     return out
141
142 def rho_estimate(n1, n2):
143     #  $\rho = |r_{01} \cdot r_{12}|$  (垂直入射近似)
144     r01 = (N0 - n1) / (N0 + n1)
145     r12 = (n1 - n2) / (n1 + n2)
146     return float(abs(r01 * r12))
147
148 def rho_drude(nu, n_inf, wp, gamma):
149     n2 = n_drude(nu, n_inf, wp, gamma)
150     r01 = (N0 - SI_N1) / (N0 + SI_N1)
151     r12 = (SI_N1 - n2) / (SI_N1 + n2)
152     return float(np.mean(np.abs(r01 * r12)))
153
154 def solve_silicon(wn1, R1, wn2, R2, d0, seed=SEED):
155     n_lin = 2
156     res_two, _ = fit_model(wn1, R1, wn2, R2, d0, twobeam_R,
157                           TWO_BOUNDS, TWO_DE_BOUNDS, seed)
158     theta_two = res_two.x
159     d_two, n_inf_two, wp_two, gamma_two = theta_two
160     n1_ = len(wn1)
161     r_two1 = res_two.fun[:n1_]
162     stats_two = fit_statistics(res_two, len(theta_two) + n_lin,
163                               R1, R2)
164     th1 = np.arcsin(np.sin(np.deg2rad(ANG1)) / SI_N1)
165     f0 = 2.0 * SI_N1 * d_two * np.cos(th1) / 1e4
166     harm_two = residual_harmonics(wn1, r_two1, f0)
167     rho_two = rho_drude(wn1, n_inf_two, wp_two, gamma_two)
168
169     res_airy, _ = fit_model(wn1, R1, wn2, R2, d0, airy_R,
170                             AIRY_BOUNDS, AIRY_DE_BOUNDS, seed)

```

```

169     theta_airy = res_airy.x
170     d_airy, n_inf_airy, wp_airy, gamma_airy = theta_airy
171     r_airy1 = res_airy.fun[:n1_]
172     stats_airy = fit_statistics(res_airy, len(theta_airy) +
n_lin, R1, R2)
173     sd_a, ci_a, _ = parameter_uncertainty(res_airy, AIRY_NAMES
, n_lin=n_lin)
174     th1_airy = np.arcsin(np.sin(np.deg2rad(ANG1)) / SI_N1)
175     f0_airy = 2.0 * SI_N1 * d_airy * np.cos(th1_airy) / 1e4
176     harm_airy = residual_harmonics(wn1, r_airy1, f0_airy)
177     rho_airy = rho_drude(wn1, n_inf_airy, wp_airy, gamma_airy)
178
179     return {
180         "d0": d0,
181         "two_beam": {
182             "theta": theta_two.tolist(), "names": TWO_NAMES,
183             "stats": stats_two, "rho": rho_two,
184             "harmonics": {str(k): v for k, v in harm_two.items
()}},
185         },
186         "airy": {
187             "theta": theta_airy.tolist(), "names": AIRY_NAMES,
188             "stats": stats_airy, "rho": rho_airy,
189             "harmonics": {str(k): v for k, v in harm_airy.
items()}},
190         "sd": sd_a.tolist(), "ci": ci_a.tolist(),
191         },
192         "f0": f0,
193         "delta_aic": stats_two["AIC"] - stats_airy["AIC"],
194         "delta_bic": stats_two["BIC"] - stats_airy["BIC"],
195         "n_data": stats_airy["n_data"],
196     }
197
198
199 def solve_sic(wn1, R1, wn2, R2, d0, seed=SEED):

```

```

200     # 与问题二同口径: Sellmeier 单振子 n1、n2 固定 2.65, 双光
201     束 vs Airy 仅差多光束高阶项
202     d_half, step = 0.5, 0.01
203     bounds = [(max(0.5, d0 - d_half), d0 + d_half)] + list(
204         SIC_SELLMEIER1_BOUNDS)
205
206     beta0_two = de_init_beta(wn1, R1, wn2, R2, d0, "sellmeier1",
207         N2_SIC, ANGLES,
208         SIC_SELLMEIER1_BOUNDS, seed=seed)
209     fit_two, prof_two = profile_scan_d(wn1, R1, wn2, R2, d0,
210         beta0_two, "sellmeier1",
211         N2_SIC, ANGLES, bounds,
212         d_half=d_half,
213         step=step, seed=seed)
214     theta_two = fit_two.theta.tolist()
215     d_two = float(fit_two.d)
216     B2, C2 = [float(v) for v in fit_two.beta]
217     stats_two = fit_statistics(fit_two.res, fit_two.res.x.size
218         + fit_two.n_lin, R1, R2)
219     r_two1 = fit_two.residual1
220     n1m = float(n_sellmeier1(wn1, B2, C2).mean())
221     th1 = np.arcsin(np.sin(np.deg2rad(ANG1)) / n1m)
222     f0 = 2.0 * n1m * d_two * np.cos(th1) / 1e4
223     harm_two = residual_harmonics(wn1, r_two1, f0)
224     rho_two = rho_estimate(n1m, N2_SIC)
225
226     beta0_airy = de_init_beta(wn1, R1, wn2, R2, d0, "
227         airy_sellmeier1", N2_SIC, ANGLES,
228         SIC_SELLMEIER1_BOUNDS, seed=seed
229     )
230     fit_airy, prof_airy = profile_scan_d(wn1, R1, wn2, R2, d0,
231         beta0_airy, "airy_sellmeier1",
232         N2_SIC, ANGLES,
233         bounds, d_half=d_half,
234         step=step, seed=seed)

```

```

225     theta_airy = fit_airy.theta.tolist()
226     d_airy = float(fit_airy.d)
227     Ba, Ca = [float(v) for v in fit_airy.beta]
228     stats_airy = fit_statistics(fit_airy.res, fit_airy.res.x.
size + fit_airy.n_lin, R1, R2)
229     r_airy1 = fit_airy.residual1
230     sd_beta, ci_beta, _ = parameter_uncertainty(fit_airy.res,
["B", "C"],
231                                                     n_lin=fit_airy
.n_lin)
232     n1ma = float(n_sellmeier1(wn1, Ba, Ca).mean())
233     th1a = np.arcsin(np.sin(np.deg2rad(ANG1)) / n1ma)
234     f0_airy = 2.0 * n1ma * d_airy * np.cos(th1a) / 1e4
235     harm_airy = residual_harmonics(wn1, r_airy1, f0_airy)
236     rho_airy = rho_estimate(n1ma, N2_SIC)
237
238     return {
239         "d0": d0,
240         "two_beam": {
241             "theta": theta_two, "names": SIC_NAMES_TWO,
242             "stats": stats_two, "rho": rho_two,
243             "harmonics": {str(k): v for k, v in harm_two.items
()}},
244         "d_ci": prof_two["d_ci"], "d_std": prof_two["std_d
"],
245     },
246     "airy": {
247         "theta": theta_airy, "names": SIC_NAMES_AIRY,
248         "stats": stats_airy, "rho": rho_airy,
249         "harmonics": {str(k): v for k, v in harm_airy.
items()}},
250         "d_ci": prof_airy["d_ci"], "d_std": prof_airy["
std_d"],
251         "sd": [prof_airy["std_d"]] + sd_beta.tolist(),
252         "ci": [prof_airy["d_ci"]] + ci_beta.tolist(),

```

```

253     },
254     "f0": f0,
255     "delta_aic": stats_two["AIC"] - stats_airy["AIC"],
256     "delta_bic": stats_two["BIC"] - stats_airy["BIC"],
257     "n_data": stats_airy["n_data"],
258 }
259
260 def main():
261     root, cfg = load_config()
262     raw = cfg["data"]["raw_files"]
263     result_dir = root / cfg["result"]["dir"]
264     result_dir.mkdir(parents=True, exist_ok=True)
265
266     print("=" * 60)
267     print("Question 3: silicon epi thickness via multi-beam (
268     Airy) model")
269     print("=" * 60)
270
271     print("\n[1] read & preprocess (Si wafer: f3=10deg, f4=15
272     deg)")
273     spec = {}
274     for key, path in [("f3", raw["f3"]), ("f4", raw["f4"])] :
275         wn, R = load_spectra(root / path)
276         wn_p, R_p = preprocess_si(wn, R, SI_CUTOFF)
277         spec[key] = (wn_p, R_p)
278         print(f"  {key}: {len(wn)} pts -> cutoff {SI_CUTOFF:.1
279         f} cm-1 -> {len(wn_p)} pts")
280         save_csv(result_dir / f"q3_processed_{key}.csv",
281                 "wavenumber_cm-1,reflectance_%", [wn_p, R_p])
282
283     wn1, R1 = spec["f3"]
284     wn2, R2 = spec["f4"]
285
286     print("\n[2] FFT thickness init")
287     d0 = fft_init_d(wn1, R1, n_avg=N_SI_AVG, theta0_deg=ANG1,

```

```

search_lo=SI_CUTOFF)
285     print(f"    d0 = {d0:.3f} um")
286
287     print("\n[3] Si wafer: two-beam vs Airy inversion &
decision")
288     si = solve_silicon(wn1, R1, wn2, R2, d0)
289     tb, ay = si["two_beam"], si["airy"]
290     print(f"    two-beam: d={tb['theta'][0]:.4f} um    RMSE={tb['
stats']['RMSE']:.4f}    "
291           f"R2={tb['stats']['R2']:.4f}    AIC={tb['stats']['AIC
']:.2f}    rho={tb['rho']:.4f}")
292     print(f"    two-beam harmonics: H2={tb['harmonics'].get
('2',0):.2f}    "
293           f"H3={tb['harmonics'].get('3',0):.2f}")
294     print(f"    airy:          d={ay['theta'][0]:.4f} um    RMSE={ay['
stats']['RMSE']:.4f}    "
295           f"R2={ay['stats']['R2']:.4f}    AIC={ay['stats']['AIC
']:.2f}    rho={ay['rho']:.4f}")
296     print(f"    airy harmonics: H2={ay['harmonics'].get('2',0)
:.2f}    "
297           f"H3={ay['harmonics'].get('3',0):.2f}")
298     print(f"    delta_AIC(two-airy)={si['delta_aic']:.2f}
delta_BIC={si['delta_bic']:.2f}")
299     print(f"    airy theta = {np.round(ay['theta'], 5)}")
300     print(f"    airy d 95%CI = [{ay['ci'][0][0]:.4f}, {ay['ci
']][0][1]:.4f}"]")
301
302     save_json(result_dir / "q3_silicon.json", si)
303
304     print("\n[4] SiC wafer (f1/f2): two-beam vs Airy inversion
& decision")
305     spec_sic = {}
306     for key, path in [("f1", raw["f1"]), ("f2", raw["f2"])]:
307         wn, R = load_spectra(root / path)
308         wn_p, R_p, cutoff, _, _ = preprocess(wn, R, **

```

```

SIC_PREPROC)
309     if not (700.0 <= cutoff <= 1600.0):
310         cutoff = 1000.0
311         wn_p, R_p, _, _, _ = preprocess(wn, R, cutoff=
cutoff, **SIC_PREPROC)
312         spec_sic[key] = (wn_p, R_p)
313         print(f"  {key}: {len(wn)} pts -> cutoff {cutoff:.1f}
cm-1 -> {len(wn_p)} pts")
314         save_csv(result_dir / f"q3_processed_{key}.csv",
315                 "wavenumber_cm-1,reflectance_osc_%", [wn_p,
R_p])
316
317     wn1s, R1s = spec_sic["f1"]
318     wn2s, R2s = spec_sic["f2"]
319     d0s = fft_init_d(wn1s, R1s, n_avg=2.6, theta0_deg=ANG1,
search_lo=1500.0)
320     print(f"  d0 = {d0s:.3f} um")
321
322     sic = solve_sic(wn1s, R1s, wn2s, R2s, d0s)
323     tb_s, ay_s = sic["two_beam"], sic["airy"]
324     print(f"  two-beam: d={tb_s['theta'][0]:.4f} um  RMSE={
tb_s['stats']['RMSE']:.4f}  "
325           f"R2={tb_s['stats']['R2']:.4f}  AIC={tb_s['stats']['
AIC']:.2f}  rho={tb_s['rho']:.4f}")
326     print(f"  two-beam harmonics: H2={tb_s['harmonics'].get
('2',0):.2f}  "
327           f"H3={tb_s['harmonics'].get('3',0):.2f}")
328     print(f"  airy:      d={ay_s['theta'][0]:.4f} um  RMSE={
ay_s['stats']['RMSE']:.4f}  "
329           f"R2={ay_s['stats']['R2']:.4f}  AIC={ay_s['stats']['
AIC']:.2f}  rho={ay_s['rho']:.4f}")
330     print(f"  airy harmonics: H2={ay_s['harmonics'].get('2',0)
:.2f}  "
331           f"H3={ay_s['harmonics'].get('3',0):.2f}")
332     print(f"  delta_AIC(two-airy)={sic['delta_aic']:.2f}

```



```

333     delta_BIC={sic['delta_bic']:.2f}")
334     print(f"    airy theta = {np.round(ay_s['theta'], 5)}")
335     print(f"    airy d 95%CI = [{ay_s['ci'][0][0]:.4f}, {ay_s['ci'][0][1]:.4f}]"
336           f"    ")
337
338     save_json(result_dir / "q3_sic.json", sic)
339
340     print("\nDone. results -> outputs/result/.")
341
342 if __name__ == "__main__":
343     main()

```